



*Asociación Profesional del Cuerpo Superior
de Sistemas y Tecnologías de la Información
de la Administración del Estado*

Temas Específicos para la preparación de la Oposición al Cuerpo Superior
de Sistemas y Tecnologías de la Información de la Administración del
Estado.

TEMAS ESPECÍFICOS II: Tecnología básica

Tema 67. Entorno de desarrollo JAVA

Editado por ASTIC en 2017 a partir de los temas numerados anteriormente como:

- **Tema 60. Entorno de desarrollo Java para software con distribución.**

AUTOR: Gonzalo Agudo del Barrio

Actualizado 2014

- **Tema 116. Arquitectura de desarrollo en la WEB (III). Tecnología Java**

AUTOR: Gonzalo Agudo del Barrio

Actualizado 2014

ÍNDICE

1.	Notas del autor	3
2.	Introducción	3
2.1.	¿Qué es Java: lenguaje de programación, plataforma, o ambas cosas?	3
2.2.	Los entornos de desarrollo y ejecución Java	4
2.3.	De Java 1.0 a Java 8	5
3.	Empaquetado de aplicaciones	8
4.	Distribución manual o tradicional	9
5.	Distribución de software mediante applets	10
6.	Distribución de aplicaciones mediante WebStart	11
7.	Java Plug-in	12
8.	Ejecución de aplicaciones nativas	14
8.1.	Compiladores nativos de Java	14
9.	Interfaces de programación para entornos de ventanas	14
9.1.	AWT (Abstract Window Toolkit)	15
9.2.	Swing	15
9.3.	SWT (Standard Widget Toolkit) y JFace	16
9.4.	JavaFX	16
10.	Principales librerías	18
10.1.	Lang y Util	18
10.2.	Sistema de entrada/salida: librerías io y nio	18
10.3.	Acceso a bases de datos: JDBC (Java Database Connectivity)	19
10.4.	JNDI (Java Naming and Directory Interface)	19
10.5.	JNI (Java Native Interface)	20
10.6.	JAXP (Java API for XML Processing)	20
11.	Cuestiones sobre seguridad	20
12.	Entornos de desarrollo integrado (IDEs) Java	21
13.	Desarrollo en Java	21
14.	Arquitectura de una aplicación web	22
14.1.	¿Qué es una aplicación web?	23
14.2.	La arquitectura multicapa	23
14.3.	El patrón MVC	24
15.	Java Platform Enterprise Edition	25
15.1.	Arquitectura Java EE	27
15.2.	Componentes Java para el desarrollo web	28
15.3.	Contenedores	33
15.4.	Servicios Java EE	34
15.5.	Empaquetamiento e instalación de aplicaciones Java EE	41
15.6.	Servidores de aplicaciones Java EE	43
16.	Frameworks de desarrollo web en Java	44
16.1.	¿Qué es un framework de desarrollo?	44
16.2.	Apache Struts	45

16.3.	Spring Framework	47
16.4.	Hibernate ORM.....	48
16.5.	JSF – JavaServer Faces	50
16.6.	Apache Tapestry	51
17.	Certificaciones profesionales Java EE	52
18.	Casos de éxito en la Administración General del Estado	53
19.	Referencias.....	55

1. Notas del autor

El presente tema no pretende ser una guía técnica y detallada de todo lo que rodea a Java sino una visión general y actualizada del contexto actual en el que se encuentra la tecnología Java. Se pretende explicar los términos, nombres y productos más importantes relacionados con el mundo Java. No se profundiza en el lenguaje de programación en sí, ya que no es el alcance de estos temas.

La revisión de septiembre de 2014 parte de la base del tema anterior elaborado por Antonio Carrasco Grande. Se ha actualizado el tema con las últimas novedades, tratando de no cambiar radicalmente su redacción (para beneficio del opositor) sino completar ciertos apartados y ampliar otros nuevos con las principales novedades.

2. Introducción

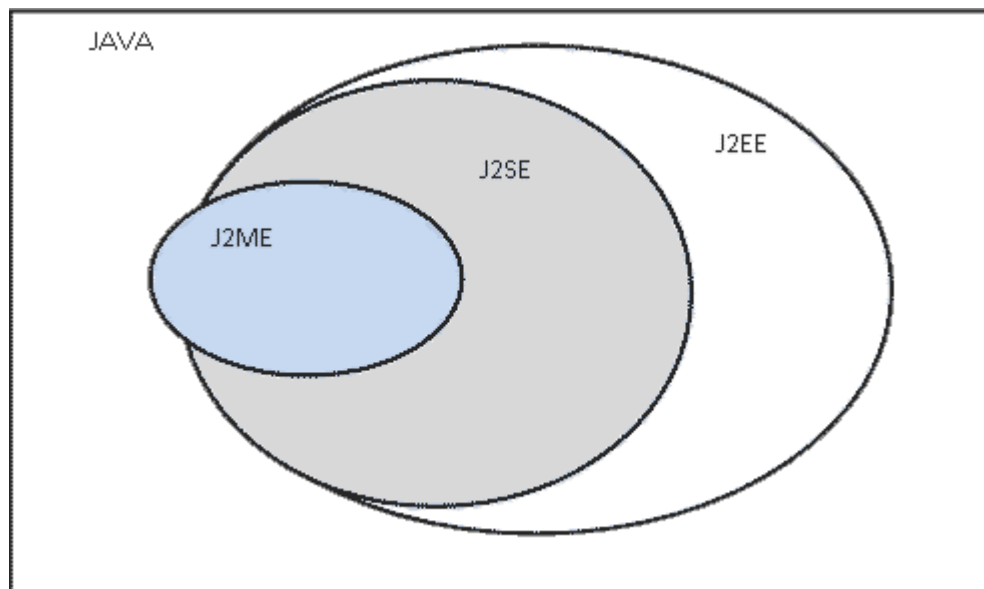
2.1. ¿Qué es Java: lenguaje de programación, plataforma, o ambas cosas?

El término Java en la actualidad no se utiliza solo para referirse al lenguaje de programación original, sino que se emplea también para referirse a toda una Plataforma. Oracle (actual propietario de Java) define Java como un lenguaje de programación y una Plataforma (separa por decir así la existencia del lenguaje de programación y de la Plataforma si bien deben convivir juntas el uno de la otra).

Podemos definir la Plataforma Java (Java Platform) como un conjunto de tecnologías (servidores, aplicaciones, especificaciones y librerías o APIs) que permiten desarrollar y ejecutar aplicaciones desarrolladas con el lenguaje de programación Java.

Oracle ha diseñado diferentes ediciones de la Plataforma Java (se suele utilizar también el término 3 plataformas):

- **Java Platform, Standard Edition (Java SE):** permite desarrollar y desplegar aplicaciones Java para su ejecución en diferentes máquinas.
- **Java Platform, Enterprise Edition (Java EE):** permite desarrollar y ejecutar aplicaciones Java en arquitectura de N capas distribuidas.
- **Java Platform, Micro Edition (Java ME):** permite desarrollar y ejecutar aplicaciones en dispositivos con recursos restringidos como teléfonos móviles, electrodomésticos etc. No se debe confundir con el término Embedded Java o Java embebido. Este término hace referencia a aplicaciones embebidas en memorias, las cuales pueden haberse desarrollado tanto en Java ME como en Java SE.



En la imagen anterior vemos, de forma orientativa, como J2EE “expande” a J2SE, mientras que J2ME “recorta” a J2SE al tiempo que tiene una fracción de contenido diferenciada exclusiva de J2ME (notar que la terminología utilizada en la imagen es la anterior ya que ahora no se utiliza J2EE sino Java EE, así como Java SE y Java ME en lugar de J2SE y J2ME). En realidad hablar de expansiones y recortes no es correcto, porque cada distribución es en sí misma distinta puesto que están concebidas con distintas finalidades. Por tanto, no puede decirse que sean expansiones o recortes, pero de forma coloquial muchas veces se interpreta así.

En este tema se verán los principales aspectos de Java SE enfocándonos en la distribución de software en máquinas cliente (de usuario) y la edición Java EE orientada a aplicaciones web sin necesidad de instalación de Java en cliente (se ejecutan en el servidor).

Como se remarca en la propia definición de la Plataforma Java, hay dos entornos bien diferenciados en Java, el entorno de desarrollo y el entorno de ejecución el cual se explican a continuación.

2.2. Los entornos de desarrollo y ejecución Java

Java como lenguaje de programación es un lenguaje:

- de propósito general: puede ser usado para varios propósitos como aplicaciones varias, sistemas operativos etc.
- de alto nivel: su programación se abstrae de los detalles de la máquina donde se va a ejecutar.
- orientado a objetos: se programan objetos para representar y modelar los datos del negocio.
- compilado e interpretado: todo programa debe compilarse, generando un bytecode el cual es interpretado en una máquina virtual.
- fuertemente tipado: no se permiten violaciones de los tipos de datos.

Pero como ya hemos dicho Java no es un simple lenguaje de programación: es también un entorno de ejecución que se ve influido por una filosofía de desarrollo determinada y las decisiones que se tomaron durante su diseño. Java nació en 1991 cuando un grupo de ingenieros de Sun Microsystems diseñaban un nuevo lenguaje de programación para sistemas empotrados. El objetivo fundamental de los ingenieros de Sun liderados por James Gosling era crear un lenguaje genérico que permitiese desarrollar programas que pudieran ejecutarse en cualquier sistema, con independencia de la CPU. Se quería huir del problema producido por el entorno tecnológico y los continuos cambios de CPU que obligaban a reprogramar todas las aplicaciones. Por ello, se diseñó un entorno basado en un lenguaje de alto nivel que al compilarse no generara código máquina para una CPU concreta sino un código binario intermedio que posteriormente se interpretara en cada CPU en lo que se denominó una máquina virtual Java o JVM (Java Virtual Machine).

Si bien es cierto que Java se ha conocido desde sus inicios por la frase “programar una vez, ejecutar en cualquier parte” (WORA Write Once Run Anywhere), también es cierto que, debido a la propia evolución del entorno, hoy por hoy, esto sólo es cierto cuando se cumplen ciertas restricciones en cuanto a la compatibilidad entre el entorno de desarrollo utilizado y la máquina virtual instalada en las plataformas en las que se requiere ejecutar el código objeto generado (“*bytecode*”). Estos posibles problemas a la hora de la ejecución de un programa vienen derivados de la utilización de librerías con diferentes versiones que forman parte del propio lenguaje (la API de Java) o de extensiones a esta API desarrolladas por empresas o por el propio desarrollador. Por ello, y aunque, como veremos existe una solución para estos problemas, el estudio de las tecnologías que envuelven el desarrollo y la distribución de aplicaciones Java es una cuestión que merece cierta consideración técnica.

Históricamente, cada entorno de desarrollo Java (JDK - Java Development Kit) ha tenido un entorno de ejecución Java (JRE – Java Runtime Environment) asociado. Un producto de software desarrollado con un JDK determinado necesita, por lo general, un JRE equivalente o superior para poder ser ejecutado en la máquina destino. Además, en ocasiones se ha roto la compatibilidad hacia atrás para determinadas clases de la biblioteca, al quedar definitivamente obsoletos los métodos o las clases previamente marcados como *deprecated*. Esto implica que determinados productos creados con un JDK concreto no pueden ser ejecutados en un entorno de ejecución de fecha posterior.

Este hecho no ha supuesto un problema insalvable ya que en una máquina pueden coexistir dos o más entornos de ejecución diferentes.

2.3. De Java 1.0 a Java 8

En este apartado vamos a ver las principales características de cada versión de Java desde la versión JDK 1.0 a la versión más reciente disponible para los desarrolladores JDK 8u20 (de septiembre de 2014), si bien la última versión disponible para la descarga por el público general es la JDK 7u67. Las últimas versiones se ponen siempre primero a disposición de los desarrolladores para dar tiempo suficiente para realizar las pruebas y certificaciones antes de que esté disponible para su descarga en el sitio web java.com. No es de interés repasar en detalle todas y cada una sino mencionar las principales novedades introducidas en cada versión y ver más en detalle la versión actual.

El primer entorno de desarrollo utilizado de forma masiva fue Java 1.0. Este entorno estaba compuesto por el JDK 1.0 y por el JRE asociado. Java 1.0 proporcionó un marco de trabajo básico para el desarrollo con Java, así como una API gráfica para el desarrollo de aplicaciones y *applets* (*componente Java incorporado en una página web html*). En realidad, desde una perspectiva actual, esta primera versión de Java tiene un interés fundamentalmente histórico ya que el JDK1.0/JRE1.0 tenía importantes limitaciones. Por ello, se hubo de esperar hasta la siguiente versión del JDK para que el producto tuviera las características que le convirtieron en un entorno de desarrollo productivo para aplicaciones reales. El lenguaje en sí mismo toma mucha de su sintaxis de Visual Basic, pero tiene un modelo de objetos más simple y elimina herramientas de bajo nivel, que suelen inducir a muchos errores, como la manipulación directa de punteros o memoria, así como la incorporación del garbage collector o recolector de basura.

En 1998 el entorno Java entró en una nueva era: Java 1.2 o Java 2. En Java 2 la interfaz gráfica de usuario estándar pasó a ser Swing¹ y se crearon las Java Foundation Classes. También se incorporaron características como la conectividad con bases de datos, la invocación remota de métodos para objetos distribuidos, un nuevo modelo de seguridad que permitía acceso controlado a los recursos del sistema y una API criptográfica, una API para manejo de gráficos en 2D, soporte para la internacionalización, colecciones, soporte para arrastrar y soltar. Se puede decir que el salto a Java 2 supuso la consagración de Java como un modelo de desarrollo maduro y moderno.

Desde el punto de vista de la distribución de aplicaciones el paso a Java 2 también supuso un cambio de modelo: a partir de la entrada en el mercado de Java 2, la versión del entorno de ejecución de Java se independizó del navegador asociado de forma que un cambio en el navegador no afectase a los entornos de Java instalados y a la inversa, una instalación de un nuevo entorno no implicase la actualización del navegador. La tecnología utilizada (denominada conexión de Java o **Java Plug-In**) permitió además la

¹ Aunque AWT se mantuvo y se siguió mejorando para garantizar la compatibilidad hacia atrás de miles de *applets* que habían sido desarrollados para esta API gráfica.

posibilidad de tener distintos entornos de ejecución para tener un mayor control sobre las distribuciones de los distintos productos.

JDK 1.3 (2000) incorporó mejoras notables de rendimiento, a la vez que amplió y mejoró notablemente la interfaz de programación gráfica Swing y las facilidades para la programación distribuida.

Desde el punto de vista de la distribución, la versión 1.3 mejoró el mecanismo de almacenamiento intermedio de *applets* y clases de soporte. También incorporó la indexación de paquetes que permite la descarga selectiva de paquetes para la ejecución de *applets*.

JDK 1.4 (2002) incorpora las aserciones, la encapsulación de excepciones, tratamiento de expresiones regulares así como nuevas funcionalidades tanto en el lado del servidor: con la definitiva introducción de XML, el soporte para arquitecturas de 64 bits, operaciones de entrada salida escalables, mejoras en las APIs de seguridad y otras características como la aceleración de gráficos 2D, extensiones de autenticación y autorización, soporte para comunicaciones seguras, soporte para Kerberos, etc.

Desde el punto de vista de la distribución, el JDK 1.4 incorporó finalmente la tecnología *Webstart* (que ya se había lanzado durante JDK 1.3, aunque como un producto independiente) lo que convirtió a Java en un entorno de desarrollo adaptado a la distribución de aplicaciones de forma centralizada y transparente, al margen de los navegadores y las tecnologías. Dedicaremos un apartado exclusivo a Java Web Start.

JDK 5.0 (2004) introduce genéricos, anotaciones, datos enumerados, número de argumentos variables etc.

JDK 6 (2006) se dota de APIs para soportar servicios web, API para el compilador, mejoras en la interfaz gráfica.

En 2010 Oracle compró SUN Microsystems haciéndose cargo entre otros de la tecnología Java así como del sistema operativo Solaris.

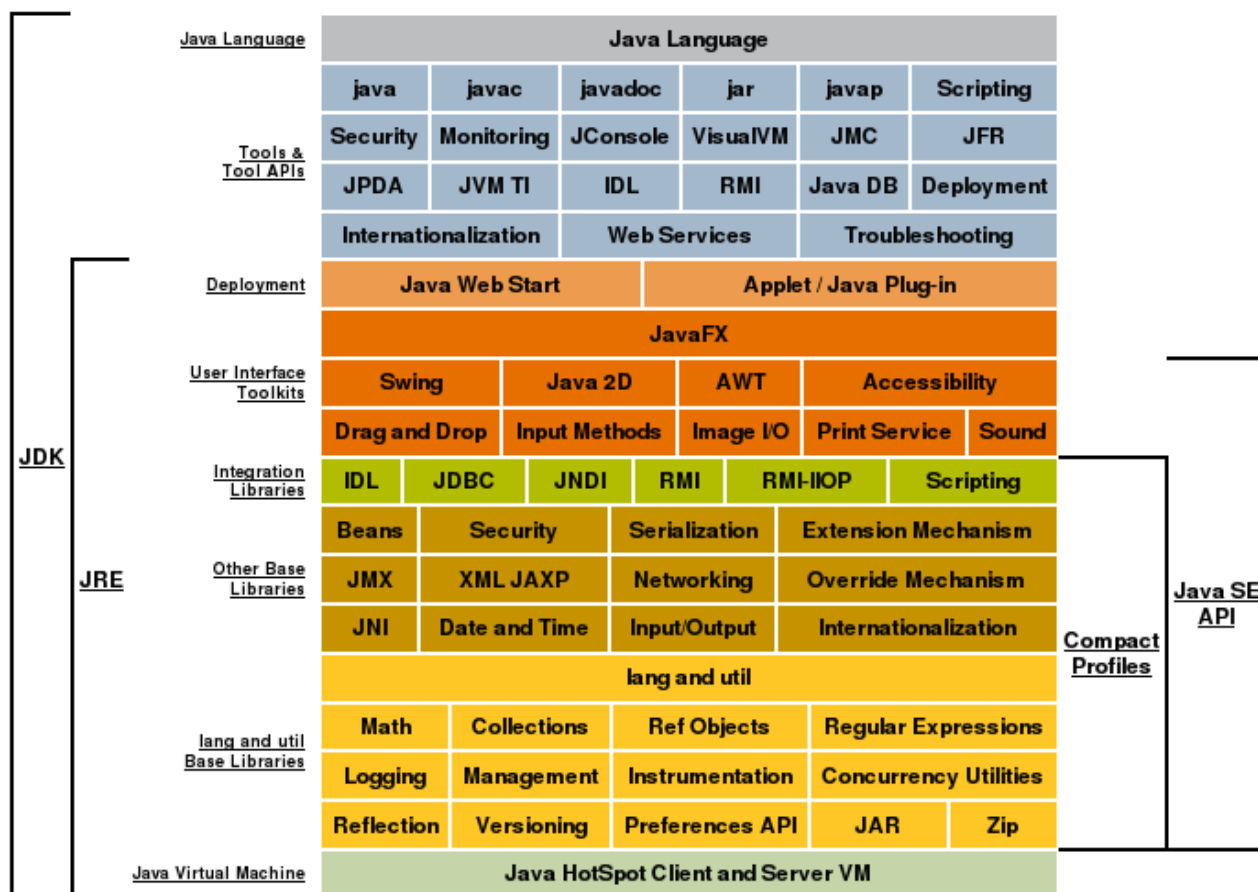
JDK 7 (2011) incorpora Strings en Switchs, multicatch, operador diamante <> así como sustanciales mejoras de seguridad siendo su última versión 7u67.

Java 8 se lanza en marzo de 2014 y como principales novedades incorpora de forma completa la librería JavaFX. A la fecha de la redacción de este tema la última versión es la JDK 8u20. Todavía no está extendido su uso y desarrollo en la actualidad, pero conviene nombrarla ya que es el futuro a corto plazo de Java.

Como se comentó anteriormente la plataforma Java (en este tema hablamos de la edición SE) es un conjunto de tecnologías que permiten desarrollar y ejecutar aplicaciones programadas en Java. Para cada edición de la plataforma hay un JDK, un JRE, documentación, test de prueba, ejemplos y especificaciones.

Un JDK contiene todo lo que contiene un JRE y además herramientas como compiladores y depuradores de errores (debuggers) necesarias para desarrollar applets y aplicaciones. Un JRE proporciona las librerías, la JVM y otros componentes para ejecutar applets y aplicaciones escritas en lenguaje de programación Java. Es importante mencionar que un JRE incluye componentes que no cumplen una especificación Java SE junto con otros componentes que sí cumplen con dichas especificaciones (JSR – Java Specification Request).

En el siguiente diagrama conceptual se muestran los componentes de la Java Platform, Standard Edition actuales y se explicarán en los siguientes apartados los más importantes.



Como vemos en la imagen los componentes Java SE se categorizan en niveles. El JDK contiene todos los niveles, mientras que el JRE contiene los niveles de despliegue, herramientas de interfaz de usuario, librerías de integración, otras librerías base, librerías base lang y útil y la JVM.

A la derecha de la imagen se puede identificar hasta donde abarca la Java SE API. La parte de arriba de las APIs (JavaFx, Java Web Start, Java Plug-in, herramientas) y la parte de abajo (JVM) es propiedad de Oracle y es Oracle la encargada de su desarrollo y mantenimiento (en consecuencia, Oracle puede estimar o desestimar su desarrollo cuando quiera). Sin embargo las APIs están basadas en especificaciones **JSR (Java SpecificationRequest)** las cuales son aprobadas dentro del **Java CommunityProcess (JCP)**. El JCP es un proceso formalizado que permite a las diferentes partes de la industria involucradas (empresas importantes del sector) participar en la definición de especificaciones JSR (que se traducen en APIs que cumplen dichas especificaciones) para que sean aceptadas en la Plataforma Java. Estas especificaciones garantizan el futuro tecnológico de Java.

Existe el proyecto **OpenJDK** el cual consiste en una implementación de software libre y abierto de la Java Platform SE, si bien no incluye Java Web Start ni Java Plug-in. Está basado en tres componentes principales: una máquina virtual, un compilador y todas las APIs que implementan las especificaciones JSR.

Como hemos visto hasta ahora, los programas Java no son tan portables como sería de desear. Y esto es, frecuentemente, con independencia de que hayamos utilizado tecnología puramente Java y conforme al estándar. Según el nivel del entorno de desarrollo que hayamos utilizado, la distribución puede ser más o menos complicada. En cualquier caso, el mantra Java *“ programe una vez, ejecute en todas partes ”* sigue siendo una utopía, por lo que, cuando estemos en la fase de análisis y desarrollo seguiremos teniendo que anticipar el entorno en el que serán instaladas o ejecutadas las aplicaciones finalmente.

Oracle Java SE no funciona en todos los sistemas operativos ni en todos los navegadores si bien, como se puede ver en la siguiente lista, está soportada un gran porcentaje de la industria:

- Sistemas operativos soportados: Solaris, Windows (Vista, 7, 8, Server 2008 y Server 2012), Linux (Ubuntu, Red Hat, Suse y Oracle Linux), Mac OS X / OS X.
- Navegadores: Internet Explorer, Firefox, Chrome, Safari (si bien solo en OX X no en iOS).

En la actualidad el gran nicho donde Java todavía no es compatible es en sistemas operativos móviles como iOS y Android. Fuera de este entorno podemos afirmar que Java puede ser una solución tecnológica candidata para el desarrollo de cualquier aplicación, con instalación en cliente estaremos hablando de Java SE, con instalación en servidor estaremos hablando de Java EE y con instalación en dispositivo móvil o periférico estaremos hablando de Java ME.

3. Empaquetado de aplicaciones

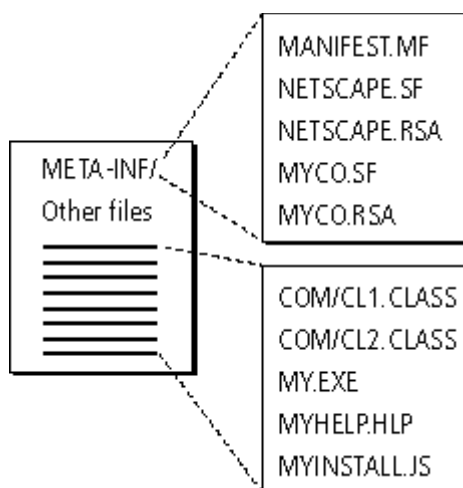
A diferencia de otros lenguajes de programación, el compilador de Java crea un fichero con extensión .class independiente por cada clase del sistema², lo que hace que, para proyectos de alta complejidad, el número de ficheros a distribuir sea muy elevado: ficheros .class, imágenes, vídeos etc.

Para facilitar la gestión, distribución, instalación y reutilización de todos estos ficheros es para lo que se han definido diferentes formatos para empaquetar aplicaciones Java:

- Formato JAR (Java Archive): formato utilizado para empaquetar applets, aplicaciones Java SE, librerías etc. Se verá en este tema.
- Formato WAR (Web Application Archive): formato utilizado para empaquetar aplicaciones web desarrolladas en Java EE.
- Formato EAR (Enterprise Archive): formato utilizado para empaquetar varios módulos (una o más aplicaciones web en formato WAR, librerías externas, o EJBs empaquetados en JAR) y desplegarlos en un servidor de aplicaciones web que cumpla las especificaciones Java EE

Un archivo JAR es un fichero en el que se reúnen las clases, imágenes, sonidos y otros ficheros necesarios para la ejecución de un *applet* o una aplicación Java. Esto permite facilitar la gestión de descargas o la distribución del programa. Un fichero JAR también puede incluir un subdirectorio de meta información llamado META-INF que contenga lo siguiente:

- Un fichero llamado MANIFEST.MF. Este fichero puede almacenar información diversa acerca de los restantes ficheros incluidos en el JAR; información como la codificación utilizada y otros parámetros locales de la aplicación, el directorio donde se debe instalar, etc.
- Una serie de ficheros con extensión .sf que contienen información relacionada con la firma de clases y aplicaciones. Debe existir uno por cada fichero firmado incluido en el JAR.
- Las firmas digitales de ficheros con una extensión que depende del formato de firma digital.



² En realidad sólo las clases concretas generan ficheros (no las clases abstractas) y, si se han definido clases internas a una clase dada, el compilador genera un fichero .class adicional por cada clase interna; esto es muy habitual cuando se definen interfaces gráficas de usuario, ya que los manejadores de eventos suelen definirse como clase internas en cada componente gráfico que debe ser gestionado, lo que rápidamente llena un proyecto de ficheros .class asociados a la interfaz gráfica de usuario.

El formato JAR no sólo cumple el objetivo de empaquetar los ficheros, sino que además los comprime utilizando un algoritmo de compresión muy extendido: el LZW (utilizado en la compresión de archivos .ZIP). Con la compresión, el tamaño de la aplicación a distribuir se reduce considerablemente. Esto es muy importante, sobre todo cuando la distribución de las clases se hace a través de Internet o la Intranet de la organización.

Junto con las versiones del JDK se proporciona una utilidad que permite la creación de estos ficheros JAR. Esta utilidad se invoca desde la línea de comandos mediante el programa “jar” y tiene las funciones típicas para añadir ficheros, extraerlos, borrarlos, listarlos, etc.

Los ficheros JAR son independientes de la plataforma y cualquier JVM los interpretará sin problemas. Este modo de empaquetar los ficheros se puede utilizar independientemente del método de distribución de software elegido, ya sea un método de distribución tradicional (es decir copiando directamente los ficheros o instándolo en la máquina donde se ejecutarán), o un sistema de distribución automática como la distribución de *applets* o *webstart*.

4. Distribución manual o tradicional

En realidad la instalación de una aplicación Java se reduce a la copia de la estructura de directorios (o los ficheros JAR) que contienen las clases de la aplicación en un directorio que esté definido en la variable de entorno CLASSPATH³ de la máquina donde se va a ejecutar. Esto es posible gracias a que los programas son compilados a un código intermedio (*bytecode*) que es interpretado por una JVM; por otra parte, esto hace necesario que exista previamente un entorno de ejecución Java (JRE).

Este método de instalación, aunque aparentemente sencillo, no es recomendable cuando el número de máquinas es muy grande (como en el caso de grandes corporaciones con miles de máquinas en las que se debe realizar una instalación), o cuando no se conocen de antemano las máquinas ni las plataformas donde se va a ejecutar un programa (como es el caso de la distribución de aplicaciones a través de Internet); en el primer caso, porque estaríamos sujetos a una carga de administración excesiva cada vez que se actualice la aplicación, se instalen nuevas máquinas, etc.; en el segundo caso sencillamente porque la administración no está centralizada ni puede estarlo. Esta es la razón por la que se han buscado nuevos modos de distribución dentro del mundo Java que veremos en los próximos apartados.

³ Aunque el intérprete de Java que se puede ejecutar desde la línea de comandos permite añadir nuevos directorios en el CLASSPATH para una ejecución concreta.

5. Distribución de software mediante applets

Un applet es un programa desarrollado en Java cuyo objetivo es proporcionar funcionalidad avanzada a una página web. Se puede incluir en una página HTML5 de forma muy sencilla mediante una etiqueta especial⁴ (<object>). En esta etiqueta se indican la clase que implementa el applet (parámetro archive) y otros atributos en los que se configura cómo debe ejecutarse. Veamos un ejemplo sencillo donde podemos observar cómo se indica la clase que contiene al applet y cómo se le pasa al entorno de ejecución los parámetros de visualización como por ejemplo la anchura y la altura del marco donde se visualizará la interfaz de usuario.

```
<object type="application/x-java-applet" height="300" width="550">
<param name="code" value="Sample" />
<param name="archive" value="Sample.jar" />
  Applet failed to run. No Java plug-in was found.
</object>
```

La característica más importante de los applets es que su instalación en la máquina que lo va a ejecutar es automática: cuando un navegador habilitado para Java detecta una etiqueta <object> de tipo applet, comprueba si el código del applet está presente en la caché de Java. Si no es así, el navegador hace una nueva petición http al servidor para descargar el código que contiene la clase a ejecutar (generalmente junto con otras clases relacionadas y agrupadas en un fichero JAR) y luego procede a cargarlo, vía el Java Plug-in, en la JVM instalada en la máquina y a ejecutarlo; en otro caso, el paso de la descarga desde el servidor se obvia, procediendo directamente a su ejecución.

Es preciso mencionar que los navegadores (originalmente Internet Explorer) ya no traen una máquina virtual propia para la ejecución de applets (por problemas legales entre Sun y Microsoft). Por ejemplo, Microsoft desarrolló la MSJVM (Microsoft JVM) la cual venía incorporada en sus ediciones de Internet Explorer para la ejecución de applets dentro del navegador sin necesidad de tener instalada una JVM en la máquina. Debido a esto en toda máquina donde se quiera ejecutar un applet es necesario tener instalado en el ordenador una JVM que como veremos viene incluida dentro del Java Plug-in.

Todo *applet* es una especialización (subclase) de la clase `java.applet.Applet`. Normalmente, al especializar la clase `Applet`, se sobrecargan (reescriben) algunos métodos de la clase original para implementar la funcionalidad específica del applet. Los métodos que normalmente se reescriben son:

- 1 `init()` – Inicializa el applet cada vez que se carga.
- 2 `start()` – Inicia la ejecución del applet cargado o cuando se visita de nuevo una página que contiene el applet.
- 3 `stop()` – Detiene la ejecución; por ejemplo, cuando el usuario abandona la página del applet o cierra el navegador.
- 4 `destroy()` – Realiza las operaciones previas a la descarga.
- 5 `paint()` – Dibuja el *applet* en el navegador.
- 6 `update()` – En conjunción con `paint()` mejora la visualización del applet.

Durante su ejecución un applet pasa por una serie de estados concretos. Estos estados son lo que podríamos denominar ciclo de vida de un applet. Estos estados son los cinco siguientes:

- Descarga del *applet*: Es el estado que antes habíamos explicado por el cual el navegador comprueba la existencia del applet en la caché de Java y, si es necesario, hace la petición http para obtenerlo.
- Carga del applet: El applet está en este estado durante el proceso de creación del objeto que concreta la subclase de `Applet` y de ejecución de los métodos `init()` y `start()`.
- Ejecución del applet: Realiza su ejecución hasta que el usuario abandone la página o cierre el navegador.

⁴ En versiones anteriores a HTML5 se utiliza la etiqueta <applet> la cual quedó obsoleta (*deprecated*) y *sustituida* por la etiqueta <object>. La versión HTML5 ya no soporta la etiqueta <applet>.

- **Applet detenido:** Cuando el usuario abandona la página el applet para su ejecución con el método `stop()`. Cuando el usuario vuelva a la página se ejecutará el método `start()` de nuevo, con lo que el applet volverá al estado de ejecución. El efecto de minimizar la ventana del navegador y posteriormente maximizar también provoca la misma secuencia `stop-start`.
- **Descarga del *applet*:** Cuando se cierra el navegador se ejecuta el método `stop()` y posteriormente el `destroy()` para liberar los recursos que estuviese utilizando el *applet*.

6. Distribución de aplicaciones mediante WebStart

Java *WebStart* es una tecnología que permite simplificar la distribución e instalación de aplicaciones Java permitiendo la ejecución de aplicaciones Java haciendo clic en un enlace de una página web desde un navegador sin realizar ninguna instalación adicional.

Con Java *WebStart* instalado en la máquina cliente (actualmente viene ya dentro del JRE que se descarga un usuario al instalar Java en un ordenador) cuando se hace clic en el enlace correspondiente, si la última versión de la aplicación no está instalada en la máquina, automáticamente se descargan todos los ficheros necesarios. De este modo podemos asegurar siempre que la máquina está ejecutando la última versión del sistema.

Las aplicaciones lanzadas con Java *WebStart* son independientes del navegador. De esa forma, tras el proceso inicial de carga y posible puesta al día, se puede cerrar el navegador y la aplicación descargada seguirá funcionando.

Java *WebStart* está diseñado para proporcionar aplicaciones con una interfaz compleja no basada en web que permitan trabajar sin estar conectado a la red y con tiempos de respuesta mejores que los de las aplicaciones web puras, a la vez que sin las desventajas de distribución, instalación y actualización de las aplicaciones tradicionales. Este esquema ofrece una solución muy conveniente para las compañías que tienen que mantener miles de máquinas actualizadas y para las que el modelo tradicional de distribución de aplicaciones se ha demostrado muy costoso.

Desde el punto de vista del usuario Java *WebStart* es transparente, no se diferencia en nada de los mecanismos para lanzar aplicaciones locales, pero en realidad lo que está ocurriendo es lo siguiente:

1. El enlace en la página manda al navegador que ejecute la tecnología Java *WebStart*.
2. Ésta determina preguntando al servidor web si la aplicación está instalada y actualizada.
3. Si no es así, se lanza la instalación o actualización de la aplicación descargando lo que sea necesario de forma transparente al usuario.

Java *WebStart* utiliza **JNLP** (Java Launching Protocol) el cual es un esquema XML que permite a los desarrolladores crear un fichero con extensión `.jnlp` para definir el mecanismo de lanzamiento. Es decir, es en ese fichero `.JNLP` donde Java *WebStart* obtiene la información de los ficheros JARs que son necesarios, su localización en el servidor, la clase principal de arranque de la aplicación así como otros atributos como el ancho y largo de la ventana.

Debe quedar claro que para que este sistema funcione debe estar instalado en la máquina cliente el software de Java *WebStart* (actualmente viene ya dentro del JRE que se descarga un usuario al instalar Java en un ordenador). Además, esta tecnología permite:

1. La convivencia de múltiples entornos de ejecución Java (JRE) en la misma máquina.
2. La implementación de políticas de seguridad basadas en código firmado.
3. La definición de políticas de seguridad para acceso a recursos locales (*sandboxing*).
4. La realización de actualizaciones incrementales: si los cambios en una versión afectan sólo al 10% de las clases de la aplicación, sólo se bajará y actualizará ese 10%.
5. Una integración completa con el escritorio de forma que el usuario no percibe la diferencia que supone lanzar una hoja de cálculo mediante una tecnología tradicional y una basada en Java *WebStart*. En un entorno de intranet (donde las aplicaciones residen en un servidor de la red local) el retraso percibido debido a las operaciones de actualización es mínimo.

Por todo esto, Java *WebStart* puede ser la tecnología en la que basarse para poder distribuir aplicaciones con interfaces de usuario complicadas que necesitan instalación de software en cliente, sin el tremendo coste de la distribución y gestión tradicional.

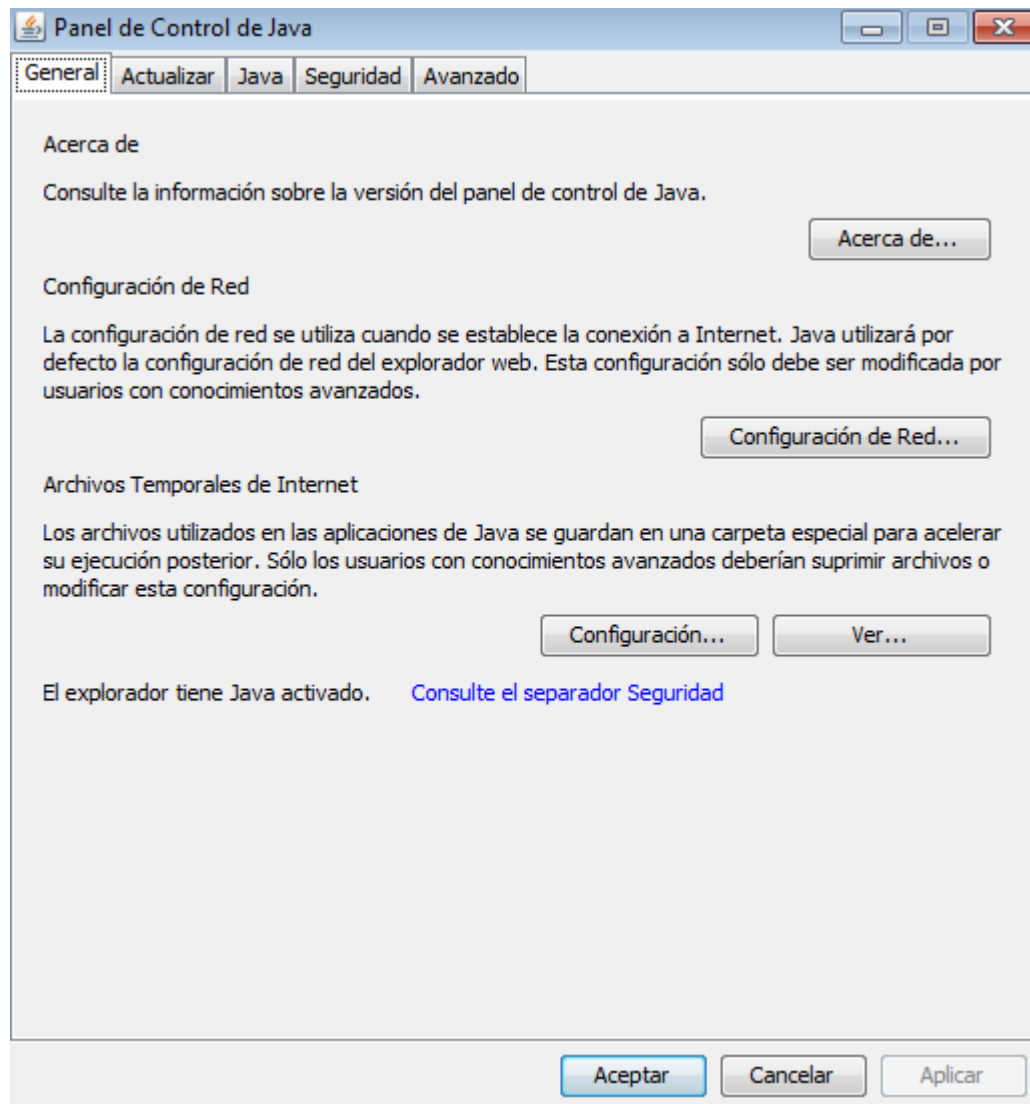
7. Java Plug-in

Java Plug-in no es más que un puente entre los navegadores instalados en el sistema y los distintos entornos de ejecución de java (JRE) que se hayan instalado en la máquina cliente. Esto permite a las empresas distribuir *applets* o *aplicaciones mediante WebStart* desarrollados con las últimas versiones y capacidades de la plataforma Java sin preocuparse por la compatibilidad con la versión del entorno de ejecución original del navegador.

Sun Microsystems introdujo la tecnología *JavaPlug-in* en 1998 mucho antes de que Java se divorciase de algunos de los navegadores más populares. Desde entonces, el *plug-in* ha sido parte de la instalación del paquete que contenía los distintos JRE que Sun y ahora Oracle han ido liberando a lo largo de los años.

Una de las cuestiones más interesantes de la tecnología del *Java Plug-in* es la capacidad que ofrece de detección de entornos de ejecución (JRE) instalados en la máquina. En este caso, el sistema permite al navegador obtener un entorno de ejecución distinto.

El elemento más característico de *Java Plug-in* es el panel. Esta es una aplicación Java que permite configurar en el sistema distintos aspectos relacionados con la actividad del *plug-in*, como la actividad de la caché, la presencia de la consola Java, la acción del compilador *just-in-time*, la gestión de certificados y la seguridad etc.



8. Ejecución de aplicaciones nativas

Una de las características más importantes de Java es que el código fuente de una aplicación escrita en este lenguaje puede ser compilado sin cambios en cualquier máquina de cualquier fabricante, pero lo más revolucionario es que el código compilado en una máquina determinada puede ser ejecutado sin ser recompilado en cualquier otra máquina para la que exista una JVM, sea cual sea la arquitectura y tecnología de esa máquina. Esto es debido a que, como hemos mencionado anteriormente, la compilación no genera código ejecutable para el hardware específico, sino que genera un código binario (*bytecode*) que es interpretado por un entorno de ejecución Java (JRE). Este modo de trabajo es lo que permite una completa independencia de la plataforma. Sin embargo, no debemos olvidar que debe existir una JRE disponible y de la versión adecuada en la plataforma en la que se quiera ejecutar las aplicaciones Java.

8.1. Compiladores nativos de Java

La portabilidad se convierte en una de las grandes ventajas del lenguaje pero lógicamente esta característica no es gratuita. El hecho de que el entorno de ejecución tenga que interpretar el *bytecode* antes de ejecutarlo ralentiza la ejecución y esto, para determinados entornos, puede ser un inconveniente decisivo: entornos de tiempo real (simuladores etc., Java ofrece menos rapidez que otros lenguajes de programación como C++).

Para solucionar este problema de rendimiento, algunos fabricantes propusieron el desarrollo de compiladores nativos de Java. Los compiladores nativos permiten a las aplicaciones escritas en Java ser compiladas directamente al código máquina nativo de un entorno concreto (el binomio hardware y sistema operativo). Por supuesto, esta solución eliminaba la independencia de la plataforma del código compilado a cambio de:

1. Proporcionar un rendimiento de ejecución supuestamente mejor y en ocasiones con una mejora absolutamente fundamental.
2. Ser innecesaria la instalación de un JRE en las máquinas que debían ejecutar la aplicación.

Por otra parte, es importante tener en cuenta ciertos aspectos antes de lanzarnos a la utilización de este tipo de compiladores ya que, aunque los compiladores nativos de Java tienen ya un nivel de madurez importante, no siempre obtienen mejores rendimientos que el código *bytecode* y esto es debido a que existen ciertas optimizaciones que las máquinas virtuales pueden realizar y que no se pueden implementar en el código nativo. De hecho, no está demostrado en la práctica que el *bytecode* sea intrínsecamente más lento que el código nativo; las diferencias más importantes están asociadas no al *bytecode* en sí mismo, sino al hecho de que la máquina virtual realiza toda una serie de comprobaciones de integridad del programa que ralentizan su funcionamiento: las llamadas retardadas, la gestión automática de memoria, la utilización de recursos por mecanismos uniformes que en muchos casos hacen subir excesivamente el consumo en memoria, etc. Estas son características del sistema por las que hay que pagar cierto precio.

En definitiva, antes de decantarse por un compilador nativo se recomienda hacer comparaciones con el entorno estándar y con otros compiladores nativos del mercado y ver cuáles son los resultados. Dependiendo de nuestras restricciones y estos resultados podremos adoptar la decisión más adecuada.

Hay diferentes tipos de compiladores nativos. Los más importantes a la hora de tomar una decisión son los llamados compiladores AOT (Ahead-Of-time) y los compiladores JIT (Just in Time). La principal diferencia entre unos y otros es que los compiladores AOT compilan a código máquina nativo antes de la ejecución de la aplicación, mientras que los JIT compilan durante la ejecución. Dependiendo del tipo de aplicación puede convenir más un compilador u otro en caso de querer compilar a código nativo.

9. Interfaces de programación para entornos de ventanas

Una interfaz de programación para entornos de ventanas es un conjunto de clases que permiten crear y gestionar ventanas, botones, elementos gráficos de la interfaz de usuario para cada sistema. Aunque, como hemos dicho antes, Java nació en el mundo de los sistemas empujados, desde su primera versión

incorporó clases y librerías para cubrir las necesidades que se iban a exigir a un lenguaje de programación de propósito general en este ámbito. Actualmente, Java tiene tres grandes colecciones de componentes (AWT, Swing y JavaFx) que facilitan la labor del programador con distintos niveles de versatilidad, potencia, sencillez y consumo de recursos del sistema.

9.1. AWT (Abstract Window Toolkit)

El paquete `java.awt` es una biblioteca para el desarrollo de interfaces de usuario gráficas (GUI) y poder gestionar entornos de ventanas desde aplicaciones Java.

AWT incluye:

- Un conjunto de componentes nativos de interfaz de usuario: es decir para sistemas operativos distintos los componentes (por ejemplo, la pantalla- window) son de apariencia diferente. Implica que las interfaces desarrolladas con AWT se ven diferentes en un sistema operativo que en otro. Este hecho es una de las principales pegas de los desarrolladores.
- Un modelo de manejo de eventos (event handler).
- Herramientas para imágenes, gráficos.
- Diferentes estilos de diseño (layouts).
- Clases para transferir datos de entrada y salida y poder cortar y pegar desde el portapapeles (clipboard) nativo del ordenador.

A pesar de sus limitaciones el paquete se sigue manteniendo en todas las versiones de Java siendo la base de otros paquetes como Swing que veremos más adelante.

9.2. Swing

Swing es, en realidad, el nombre clave del proyecto que desarrolló los nuevos componentes de la JFC (Java Foundation Classes) como extensión a los componentes estándar del AWT incluido en la versión 1.1 del JDK. Sin embargo, este nombre ha venido a ser conocido de forma generalizada para referirse también a los resultados de este proyecto; es decir al conjunto de componentes de nueva generación (flexibles, 100% Java y basados en la nueva arquitectura de eventos).

Los objetivos de Swing eran:

- Ser 100% Java para promover la consistencia de las GUI entre plataformas y para facilitar el mantenimiento.
- Ofrecer una única API capaz de ofrecer múltiples aspectos de forma que desarrolladores y usuarios no tuvieran que estar condicionados por una cuestión más relacionada con la estética que con la funcionalidad.
- Capacitar al sistema de un modelo dirigido por eventos que no implicase tener que tratar con él en otras APIs de nivel superior.
- Mantener la adherencia a los principios de diseño de JavaBeans para asegurar que los componentes se podían utilizar sin problemas en las herramientas y los entornos de desarrollo para Java.
- Ofrecer compatibilidad con las APIs de AWT donde hubiera elementos comunes, para facilitar el acceso de los programadores que utilizasen AWT.

Swing está incluida bajo el paquete `javax.swing` y amplía la funcionalidad de AWT con muchos más tipos de componentes gráficos y, gracias a su implementación 100% Java, proporciona una mayor versatilidad para que el programador pueda ampliar y ajustar las funcionalidades de dichos componentes (además de no depender de las implementaciones nativas de los sistemas de gestión de ventanas). Estas son las principales diferencias con AWT, aunque hay otras como la mayor escalabilidad y el mejor rendimiento del sistema gráfico.

La arquitectura de Swing está basada en un patrón de diseño muy conocido: el patrón modelo-vista-controlador (MVC). Esto permite independizar bastante la lógica de la aplicación diseñada de la interfaz concreta, de forma que es más sencillo hacer cambios en una u otra con un menor impacto en la totalidad del conjunto. Casi todos los componentes contenedores de Swing tienen un modelo (funciones) asociado.

Por ejemplo, un botón (Jbutton) tiene un modelo asociado que almacena el estado del botón (cuál es su tecla asociada, si está habilitado, si está seleccionado o presionado, etc.); una lista (Jlist) tiene dos modelos asociados: uno que mantiene el contenido de la lista y otro que lleva el control de la selección actual de la lista. Por lo general, no es necesario que el programador sepa cómo gestionar el modelo del componente; la cuestión de la existencia de los modelos, pues, no se justifica en realidad porque el programador los utilice directamente, sino porque ofrecen una flexibilidad única al determinar cómo se almacenan los datos y cómo se recuperan. Por ejemplo, si un equipo de desarrollo está diseñando una aplicación que incluye una hoja de cálculo que maneja datos de una matriz triangular, el programador puede crear su propio modelo de tabla optimizado para este caso con un consumo de recursos mínimo.

Con Java 2, Swing empezó también a hacer uso extensivo de otra API que mejoró considerablemente muchos aspectos de la visualización en pantalla: Java2D. Esta API es un conjunto de clases que ofrecen herramientas avanzadas para gestionar gráficos 2D e imágenes, con un modelo sencillo a la par que completo.

Hoy por hoy, Swing ofrece todas las características necesarias para crear interfaces de usuario robustas y complejas siendo el paquete más utilizado para desarrollar interfaces de usuario para aplicaciones de escritorio desarrolladas con Java SE. Es una colección de componentes madura y potente. Sin embargo, su principal crítica es que las interfaces que se realizan con Swing tienden a ser relativamente proclives a consumir una gran cantidad de recursos de máquina en forma de memoria. Por ello, han surgido distintas alternativas como SWT (Estándar Widget Toolkit) y JFace de la Fundación Eclipse y Java FX (incluida ya en el JDK/JRE 8) que veremos a continuación.

9.3. SWT (Standard Widget Toolkit) y JFace

SWT es una librería de código abierto diseñada para construir interfaces gráficas de usuario por la fundación Eclipse.

SWT/JFace es similar a AWT/Swing en Java con la diferencia de que utiliza un amplio conjunto de componentes gráficos nativos, más ricos que los de AWT y más sencillos que los de Swing.

Swing tenía una cierta fama de lento. En 1998, tras la liberación de la plataforma Java 2, había sin duda problemas de rendimiento en algunas áreas. Sin embargo, cuatro años después, el rendimiento se había mejorado y estabilizado bastante. Los autores de SWT dicen que la principal razón para crear SWT era el rendimiento.

Los desarrolladores necesitaban librerías portables que proporcionaran componentes gráficos comparables a los que ofrecía cualquier herramienta desarrollada con herramientas específicas. Algunos reclamaban acceso a características específicas de cada plataforma. SWT trató de proporcionar esta funcionalidad utilizando una pequeña API. Esta API se implementó sobre diferentes plataformas con una combinación de componentes Java y componentes nativos. La implementación no era puramente Java ya que utilizaba el interfaz nativo con C (JNI) para llamar al sistema operativo desde Java. La idea de los ingenieros de Eclipse era utilizar los componentes nativos del sistema para acelerar el rendimiento de la interfaz. Finalmente, rodearon el conjunto con una paleta de componentes denominados JFace (que abstraían la implementación concreta de cada sistema operativo).

En cierto sentido SWT tiene muchos elementos en común con AWT: son tecnologías de componentes pesados; aunque SWT lo que ofrece es un conjunto de componentes más extenso. Pero en realidad tienen las mismas limitaciones.

Otros problemas con SWT tienen relación con la propia naturaleza de Java como lenguaje multiplataforma, cuya mejor baza se supone que es la portabilidad. Con SWT esta portabilidad se reduce bastante. Si consideramos que, por otra parte, según las pruebas comparativas realizadas entre la primera versión de Java 2 (JDK 1.2) y la actual (JDK 8) el rendimiento de Swing es considerablemente mejor, la conveniencia de utilizar SWT/JFace por su mejor rendimiento está claramente cuestionada.

9.4. JavaFX

JavaFX pretende sustituir a Swing como estándar de desarrollo de interfaces gráficas de usuario en Java SE. JavaFX viene ya incluido como novedad desde la versión 7u6 del JDK/JRE pero no está basado en ninguna especificación estándar JSR sino que es un producto de Oracle, por lo que Oracle puede suspender su desarrollo si lo considera oportuno. Esta situación es importante tenerlo en cuenta a la hora de elegir un nuevo desarrollo de una aplicación de escritorio haciendo uso de Swing o de JavaFx.

Entre sus mejoras está que soporta gráficos en 3D, interfaces más atractivas, efectos visuales, permite añadir JavaScript etc.

10. Principales librerías

A continuación se detallan las principales librerías de la edición Java SE.

10.1. Lang y Util

El paquete Java `java.lang` contiene clases e interfaces fundamentales estrechamente ligado al tiempo de ejecución: (Object, Enum, Class, Throwable, Error, Exception...)

El paquete Java `java.util` contiene las estructuras de datos que agregan objetos, como los tipos Collection, Set...

10.2. Sistema de entrada/salida: librerías io y nio.

El sistema de entrada/salida de un lenguaje siempre es un punto crítico en cuanto al rendimiento y escalabilidad de las aplicaciones desarrolladas con dicho lenguaje. Un programa se puede comunicar con su entorno a través de ficheros, una consola, la red, bases de datos, etc. Además ese acceso puede ser secuencial o directo, en modo binario o en modo carácter, síncrono o asíncrono, etc. Es decir, existe una gran funcionalidad que debe ser cubierta por las librerías que dan soporte a la entrada/salida.

Los diseñadores de Java crearon un gran número de clases para cubrir estos aspectos de entrada/salida. Para muchos programadores que se inician en este entorno, esta complejidad de la API de entrada/salida puede hacer que la curva de aprendizaje sea algo más pronunciada que en otros lenguajes.

La entrada/salida desde Java se realiza a través de las clases del paquete `java.io`. Dentro de este paquete aparecen jerarquías y clases dedicadas a funcionalidades más concretas. Por ejemplo:

- File. Representa un path a un fichero o directorio.
- InputStream. Representa una clase para recibir un flujo de datos de entrada.
- OutputStream. Representa una clase para generar un flujo de datos de salida.
- Reader. Clase orientada a la entrada de caracteres.
- Writer. Clase orientada a la salida de caracteres.
- ...

Así, podríamos enumerar un enorme conjunto de clases que combinadas proporcionan a Java una completa funcionalidad de entrada/salida.

Con la versión 1.4 del JDK aparece un nuevo paquete de librería de entrada/salida: `java.nio`, que tiene como principal objetivo mejorar el rendimiento de las clases del que, hasta ese momento, había sido el paquete estándar de entrada/salida: `java.io`. La principal diferencia entre `java.io` y `java.nio` es que la primera utiliza streams y la segunda blocks, a la hora de almacenar y transmitir la información, siendo la segunda opción más rápida y flexible.

En definitiva, podemos decir que, en su versión actual, Java encapsula toda la funcionalidad genérica de entrada/salida en dos paquetes `java.io` y `java.nio`. Claro que también existen librerías que cubren otras necesidades de entrada/salida específica como puede ser el acceso a bases de datos.

10.3. Acceso a bases de datos: JDBC (Java Database Connectivity)

La API `java.sql` es un paquete de clases conocido como JDBC (Java Database Connectivity) que proporciona una interfaz común para el acceso a bases de datos (generalmente relacionales) a través del lenguaje SQL (Standard Query Language). JDBC permite el acceso a un gran número de bases de datos, en realidad no sólo a bases de datos, sino a cualquier fuente de datos tabular para la que el fabricante haya proporcionado un controlador específico.

En realidad es importante entender que el paquete `java.sql` no está formado fundamentalmente por clases, sino que casi en su totalidad es un conjunto de interfaces. Los programas Java que el desarrollador escribe para acceder a una base de datos referencian a los métodos declarados por estas interfaces y serán los drivers los encargados de implementar esos métodos. De este modo se consigue completa independencia del código fuente y del sistema gestor de base de datos (aunque no olvidemos que la mayoría de los gestores de bases de datos tienen extensiones propietarias de SQL y la utilización de estas extensiones arruina la posible independencia de nuestros programas respecto del gestor de base de datos que hayamos utilizado).

En JDBC se distinguen cuatro tipos de drivers, que se numeran desde el tipo I al tipo IV.

El driver de tipo I es el bridge JDBC-ODBC. Este driver utiliza controladores ODBC para conectarse con el sistema gestor de base de datos. Por lo tanto, requiere que el controlador ODBC esté configurado correctamente en la máquina de ejecución.

La ventaja de este tipo de driver es que da acceso a prácticamente todos los sistemas de gestión de base de datos del mercado puesto que los drivers ODBC están muy extendidos. Como desventajas deben destacarse, por un lado, la existencia de tres niveles de software entre la aplicación y la base de datos (API JDBC, bridge JDBC-ODBC, driver ODBC) lo que disminuye considerablemente el rendimiento total del sistema, y por otro lado la utilización del driver ODBC (código nativo – no Java) se salta el modelo de seguridad del JRE por lo que su utilización en *applets* no está recomendada.

Driver de tipo II o driver parcialmente nativo. Este tipo de driver utiliza código Java para hacer llamadas a una API nativa que debe estar en la máquina que accede a la base de datos. Esta API nativa es la que realmente proporciona la conectividad.

La ventaja de este tipo de drivers es la rapidez ya que está utilizando código nativo de una API propietaria para una base de datos. Sin embargo, al utilizar código nativo tiene el mismo inconveniente que los de tipo I de cara a los *applets*.

Driver de tipo III o Middleware driver. Este tipo de driver utiliza los protocolos de red del JDK para conectar con un servidor intermedio. El driver JDBC traduce las peticiones a un protocolo independiente de la base de datos y las envía a dicho servidor. Éste suele ser un servidor físicamente diferenciado, aunque no es necesario. El intermediario traduce las peticiones del programa al lenguaje específico de la base de datos. Este tipo de driver no requiere ninguna instalación adicional en la máquina cliente, pero obliga a la existencia del *middleware*. La principal ventaja es que sólo requiere código Java en cliente y por lo tanto puede ser utilizado con seguridad por *applets*. La desventaja es la capa, *middleware* que añade un nivel más de software y por lo tanto provoca una disminución de rendimiento.

Driver de tipo IV o Pure Java driver. Este tipo de drivers están desarrollados exclusivamente en Java y utilizan directamente el protocolo propietario de la base de datos a la que accede. Al igual que los de tipo III no requieren software adicional en cliente y por lo tanto pueden utilizarse en *applets* y su principal ventaja está en que, al tener menos capas, ofrecen mejor rendimiento. Generalmente el fabricante de la base de datos que vayamos a utilizar nos proporcionará su driver JDBC de tipo IV (también nos proporcionará otros tipos, aunque el tipo más utilizado suele ser el 4. Un ejemplo es el driver “thin” de Oracle ampliamente utilizado.).

10.4. JNDI (Java Naming and Directory Interface)

API de Java para acceso a servicios de directorio (LDAP, DNS etc.). Permite a las aplicaciones Java clientes descubrir y buscar objetos y datos a través de un nombre. Dentro de la pila de Java SE API se

encuentra en el grupo de librerías de integración. Es utilizada por otras librerías como RMI y librerías de Java EE.

10.5. JNI (Java Native Interface)

JNI es la interfaz estándar de programación para llamar a métodos nativos escritos en otros lenguajes como C o C++.

Normalmente, un desarrollador de Java programará sus aplicaciones completamente en Java; por un lado, por comodidad y, por otro, para evitar incompatibilidades con las plataformas. Sin embargo, en algunas ocasiones el desarrollador puede considerar adecuado utilizar métodos implementados en otros lenguajes y compilados exclusivamente para una plataforma. Ejemplos de esta problemática son los siguientes:

- Cuando busquemos un mejor rendimiento: determinados procesos críticos pueden ser más rápidos si han sido desarrollados en lenguajes como C o C++.
- Cuando tengamos que utilizar características propias de una plataforma concreta.
- Cuando tengamos que utilizar librerías escritas en otros lenguajes y éstas no existan en Java.
- Cuando tengamos programas o código heredado para el que no se disponga de un conector y no sea aconsejable técnica o económicamente migrarlo a Java.

JNI nos permite cargar librerías dinámicas de la plataforma en la que nos encontremos (por ejemplo: ficheros DLL en Windows o librerías .so en UNIX) en nuestro programa Java y posteriormente acceder a los métodos y propiedades de estas librerías de forma casi transparente para el programador.

Otra de las ventajas que nos proporciona la interfaz JNI es que podemos crear aplicaciones nativas y utilizar desde estas clases desarrolladas en Java. El JNI nos permite cargar, inicializar e invocar a la máquina virtual de Java para poder crear clases Java desde nuestro programa escrito en lenguaje nativo.

Sin embargo, atendiendo a la distribución de aplicaciones no hay que olvidar que al utilizar librerías nativas en nuestros programas Java, estamos perdiendo la famosa portabilidad universal entre diferentes plataformas de nuestra aplicación Java quedando limitada a la portabilidad de las librerías nativas.

10.6. JAXP (Java API for XML Processing)

API de Java para validar y parsear documentos XML. Proporciona varias interfaces para poder parsear el documento XML en modo DOM, SAX, StAX (Streaming SAX) y una interfaz XSLT (XML Stylesheet Language for Transformations) para poder transformar el documento XML en otro tipo de documento (HTML, PDF...).

11. Cuestiones sobre seguridad

Una aplicación Java se enfrenta a los mismos problemas de seguridad que una aplicación en cualquier otro lenguaje, aunque como veremos Java implementa ciertas funcionalidades que harán más seguro el entorno de ejecución. Una aplicación nativa tiene acceso a los recursos del sistema sobre el que se ejecuta, sólo limitada por el esquema de seguridad del sistema operativo o por productos de seguridad específicos.

El lenguaje Java incluye en su diseño una serie de características que han ayudado a hacer de Java un entorno bastante seguro. Algunas de estas características son implementadas por el compilador:

- Es un lenguaje fuertemente tipado. El intento de actualizar posiciones de memoria con datos de tipo incorrecto es bloqueado por el compilador generando errores en tiempo de compilación.
- Eliminación de operaciones con punteros. Con lo que se elimina el potencial riesgo de acceso a posiciones de memoria restringidas o a la modificación de código cargado en memoria.

Durante la compilación se verifican las reglas de seguridad que hemos mencionado, pero como no podemos garantizar que el código compilado no ha sido modificado o partir de la base de que todos los compiladores hacen lo que tienen que hacer (alguien podría construirse su propio compilador que permitiese hacer

operaciones ilegales para un compilador estándar de Java), el entorno de ejecución implementa estas revisiones antes de ejecutar la clase. Con esta revisión se garantizan temas como:

1. Que no se realicen operaciones con punteros.
2. Que no existan desbordamientos que puedan permitir el acceso a zonas de memoria a las que no deberían acceder.
3. Que los tipos de los parámetros coinciden con lo que se espera.
4. Que no se está accediendo a métodos privados o protegidos desde una clase que no deba tener accesos a ellos.
5. Que el fichero compilado está bien formado.

Estas características, indudablemente, les restan oportunidades a los que con determinados trucos intentan saltar la seguridad de un programa y propone a Java como un lenguaje con el que desarrollar sistemas seguros.

12. Entornos de desarrollo integrado (IDEs) Java

Un entorno de desarrollo integrado, llamado también IDE (sigla en inglés de Integrated Development Environment), es un programa informático compuesto por un conjunto de herramientas de programación. Puede dedicarse en exclusiva a un solo lenguaje de programación o bien puede utilizarse para varios.

Un IDE incorpora habitualmente:

- Un espacio para la escritura de código con cierta ayuda interactiva para generar código y para indicar los errores de sintaxis que se cometan por parte del programador.
- La posibilidad de compilar y ejecutar el código escrito.
- La posibilidad de organizar los proyectos de programación.
- Herramientas auxiliares para programadores para detección de errores o análisis de programas (depuradores).
- Otras opciones como utilidades para pruebas, carga de librerías, etc.

Existen diversos IDEs para Java. Se citan los más utilizados en la actualidad, siendo los dos primeros sin lugar a duda los más utilizados:

1. **Eclipse:** software libre. Es uno de los entornos Java más utilizados a nivel profesional. El paquete básico de Eclipse se puede expandir mediante la instalación de plugins para añadir funcionalidades a medida que se vayan necesitando.
2. **NetBeans:** software libre. Otro de los entornos Java muy utilizados, también expandible mediante plugins. Facilita bastante el diseño gráfico asociado a aplicaciones Java.
3. **BlueJ:** software libre. Es un entorno de desarrollo dirigido al aprendizaje de Java (entorno académico) y sin un uso muy extendido a nivel profesional. Es utilizado en distintas universidades para la enseñanza de Java. Destaca por ser sencillo e incluir algunas funcionalidades dirigidas a que las personas que estén aprendiendo tengan mayor facilidad para comprender aspectos clave de la programación orientada a objetos.

13. Desarrollo en Java

Las arquitecturas web han revolucionado, sin duda, el panorama del desarrollo de software en los últimos años. Esta revolución se ha debido a diferentes causas de entre las que destacan poderosamente la explosión de Internet y sus protocolos, y el apoyo por parte de la industria a este nuevo paradigma de aplicación.

El desarrollo de sistemas corporativos basados en una arquitectura web es algo relativamente reciente. Durante los años 90, los desarrollos de aplicaciones corporativas se diseñaban con tecnología cliente/servidor para incrementar la productividad, de optimizar la inversión en hardware y de proporcionar la información donde y cuando se necesitaba.

La implantación de la tecnología cliente/servidor no se hizo sin problemas ya que dieron lugar a entornos heterogéneos y distribuidos que eran muy difíciles de integrar y sobre todo de gestionar (distribución, instalación, actualización).

La necesidad de distribuir aplicaciones a un creciente número de clientes, situados en centros dispersos en una geografía nacional, o incluso internacional, creó importantes problemas de eficiencia, eficacia y servicio durante la utilización de estas aplicaciones cliente/servidor.

El objetivo inicial de alcanzar flexibilidad, acceso rápido y seguro a los datos corporativos, a un precio razonable, seguía siendo una meta por conseguir. El coste de gestión de estos sistemas era enorme y las corporaciones seguían buscando una nueva tecnología que resolviese al menos algunos de estos problemas.

A finales de los 90, gran número de desarrollos corporativos empezaron a aprovechar los estándares, lenguajes y protocolos surgidos alrededor de la web, creándose el concepto de las arquitecturas multicapa, que se presentaban como alternativa a la arquitectura cliente/servidor clásica, que permitiría utilizar un entorno cliente ligero y, por lo tanto, evitar el problema de la distribución y mantenimiento del software de los PCs cliente.

Indudablemente estos inicios fueron duros ya que la tecnología que se estaba utilizando no había sido inicialmente pensada para lo que ahora se quería utilizar. Las empresas de software iniciaron una carrera para conseguir adaptar los protocolos, lenguajes y estándares de Internet, que parecían claramente asentados en un tipo de aplicaciones con gran cantidad de información estática e interfaces de usuario sencillas, a las necesidades de los desarrollos corporativos estratégicos.

Java es uno de estos lenguajes que aparecieron alrededor de Internet. Inicialmente, los *applets* de Java contribuyeron a enriquecer las interfaces de usuario, posteriormente, tecnologías como los *servlets* y los *JSPs*, facilitaron el desarrollo de la lógica de negocio compleja para los servidores.

Finalmente, el modelo MVC y los frameworks de desarrollo Java surgidos alrededor suyo han dado el impulso definitivo, de forma que podríamos afirmar que Java ha impulsado el desarrollo de aplicaciones con arquitectura web como ningún otro lenguaje.

La complejidad de las arquitecturas web seguía siendo bastante importante, poco a poco se definieron recomendaciones y estándares que intentaron simplificarlo. Con la publicación de la edición *Java Platform Enterprise Edition (Java EE)* se dio el impulso definitivo para la entrada de esta tecnología en el ámbito empresarial, con muchas menos dudas, ya que estos estándares sí fueron pensados para la problemática de complejos desarrollos corporativos y se ha convertido en la base para los próximos años de la arquitectura de desarrollo para la web con tecnología Java.

En este tema veremos un primer punto en el que empezaremos dando un repaso a la arquitectura de una aplicación web.

Posteriormente analizaremos la especificación Java EE desde los diferentes puntos de vista de arquitectura, estándares y servicios, y veremos algunos ejemplos de servidores de aplicaciones compatibles con las especificaciones Java EE.

Por último, analizaremos algunos de los *frameworks* de desarrollo Java más populares de los últimos años, que han potenciado enormemente el desarrollo de aplicaciones web basadas en Java frente a otras tecnologías y veremos las certificaciones profesionales y algunos casos de éxito en la Administración General del Estado (AGE).

14. Arquitectura de una aplicación web

En este apartado vamos a ver a qué llamamos aplicación web, cómo se divide esta aplicación en diferentes niveles o capas generando lo que llamamos una arquitectura multicapa para, finalmente, revisar el patrón de diseño MVC (modelo, vista, controlador) base para el diseño de aplicaciones que utilizan arquitecturas multicapa.

14.1. ¿Qué es una aplicación web?

Una posible definición de aplicación web puede ser la siguiente:

Una aplicación web es un sistema de software cuyo interfaz de usuario está basado en páginas HTML que se muestran en un navegador web y los eventos que el usuario genera (desde el navegador) se comunican con un servidor a través del protocolo HTTP.

Lo que caracteriza a una aplicación web es su arquitectura: una aplicación web difícilmente se puede desarrollar como algo monolítico sino como un conjunto de elementos de software, unos dedicados al interfaz de usuario (o a ejecutarse en la máquina cliente), otros que se ejecutan en el servidor atendiendo a las peticiones HTTP, y otros para implementar la semántica de la transacción que el cliente ha solicitado, lo que normalmente se denomina lógica de negocio.

Esta división de los elementos de software en distintas capas ha generado la especialización de lenguajes diferentes para las distintas capas de una aplicación.

Evidentemente, esto ha hecho que el desarrollo sea más complejo y que en los grupos de desarrollo deban incluirse expertos en cada uno de estos lenguajes especializados bien en la parte cliente de la aplicación o en la parte servidora.

14.2. La arquitectura multicapa

La arquitectura de desarrollo multicapa se caracteriza, como ya se ha apuntado en el apartado anterior, por dividir el sistema en distintos niveles o capas que se comunican a través de interfaces.

Surge de aplicar el patrón arquitectónico de capas (*layers pattern*)⁵, siendo un requisito de este patrón que los componentes de una capa accedan únicamente a los componentes de la capa inferior.

El desarrollo web, por sus especiales características, utiliza casi desde sus inicios un desarrollo en varias capas:

1. Capa de presentación o cliente: Es la encargada de mostrar la interfaz de usuario al cliente e interactuar con él.
2. Capa de negocio: Es la parte en la que se reciben las peticiones de usuario y se ejecuta toda la lógica transaccional de la aplicación.
3. Capa de almacenamiento de datos: Es fundamentalmente la formada por el sistema gestión de base de datos, aunque en ocasiones también incluye cierta funcionalidad de manejo de esos datos.

Esta arquitectura busca por un lado aprovechar las especiales características de los diferentes lenguajes o tecnologías de desarrollo y por otro independizar unas capas de otras tratando de conseguir que un cambio en una capa no afecte al resto de capas del sistema.

Sin embargo, una arquitectura multicapa, por si sola, no nos permitirá alcanzar estos objetivos. Se necesitará además una buena formación de los equipos de trabajo (con buenos especialistas), un buen diseño y una buena definición de las interfaces entre las capas, sólo así lo tendremos un poco más fácil.

En el desarrollo de aplicaciones corporativas, la principal virtud de aplicaciones con esta arquitectura es la necesidad de poco o ningún software en las máquinas cliente lo que significa un abaratamiento de la gestión de las empresas con gran número de empleados. Ahora, la progresiva aparición de herramientas comerciales, que facilitan la utilización de esta tecnología en el desarrollo de aplicaciones, la ha convertido en una buena alternativa para el desarrollo corporativo.

Si analizamos qué caracteriza a las aplicaciones empresariales podríamos destacar los siguientes aspectos:

⁵ Sus autores son Frank Buschmann, Regine Meunier, Hans Rohnert, Peter Sommerlad y Michael Stal, y puede consultarse junto a otros muchos patrones en el libro "*Pattern-Oriented Software Architecture Volume 1: A System of Patterns*" de los mismos autores.

1. Alta escalabilidad y rendimiento. Las aplicaciones corporativas son complejas y requieren tiempos de respuesta razonables, así como adaptarse a los picos de trabajo sin sufrir una degradación apreciable en el nivel de servicio.
2. Integridad transaccional. Las aplicaciones de gestión requieren seguridad en las transacciones que garanticen la integridad del modelo de negocio gestionado.
3. Robustez. El trabajo diario de las organizaciones depende de estos sistemas de información, por lo que se debe proporcionar una calidad de funcionamiento y unos niveles de disponibilidad adecuados.
4. Interoperatividad. Deben integrar sistemas heterogéneos y distribuidos.
5. Seguridad. La información gestionada por estos sistemas, en la mayoría de las ocasiones, contiene información sensible para la que se debe garantizar tanto su confidencialidad y como su integridad.

Esto obliga a incorporar a la arquitectura nuevos servicios a los clásicos elementos del desarrollo web tradicional. Así aparecen nuevas capas como la capa de integración o la capa de gestión sistemas transaccionales. Este conjunto de nuevas funcionalidades complica aún más el entorno y aparecen más problemas: la gestión de sistemas distribuidos, la interoperatividad de sistemas, los nuevos entornos de desarrollo, etc.

En definitiva, estas arquitecturas son complicadas y nos obligarán a tener una gran variedad de personal especializado en áreas tan distintas del desarrollo como la creación de interfaces de usuario, el desarrollo con lenguajes orientados a objetos como Java, el desarrollo de librerías de integración con sistemas propietarios y heredados.

El reto de la industria del software es desarrollar herramientas y estándares que hagan cada vez más fácil a los desarrolladores el trabajo en este complejo entorno, así como ofrecer la necesaria formación que exige este mundo. Con este objetivo aparecieron las especificaciones Java EE que veremos más adelante en este tema.

14.3. El patrón MVC

El patrón de diseño MVC es un modo de dividir la funcionalidad de una aplicación entre distintos componentes de forma ordenada, y a pesar de su amplia difusión hoy en día con las arquitecturas web, nació en el año 1980 junto al lenguaje Smalltalk en el seno de la empresa Xerox.

Esta división se hace con el objetivo de que unos componentes se encarguen de la lógica de negocio (modelo), otros del flujo de control de la aplicación (controlador) y otros de la presentación o interfaz de usuario (vista), siguiendo las características anteriormente expuestas para una arquitectura multinivel, es decir, permitiendo la utilización de entornos especializados en cada capa y minimizando el acoplamiento entre los componentes de distintas capas.

Repasemos cada uno de estos tres módulos en que el patrón MVC divide una aplicación.

En primer lugar, veamos qué es el modelo. El modelo representa los componentes que implementan el acceso a los datos de la aplicación y las reglas de negocio que gestionan su acceso. El modelo es, en definitiva, el punto por el que cualquier sistema que desee acceder a los datos de la aplicación deberá pasar.

Para el desarrollo del modelo se utilizan normalmente lenguajes de propósito general como Java con librerías de acceso a sistemas de gestión de base de datos e incluso lenguajes para desarrollar procedimientos almacenados integrados en los propios sistemas de gestión de base de datos.

Es común el uso del patrón DAO (*Data Access Object*) para el desarrollo del modelo. Mediante este patrón, se construye una capa "de acceso a datos" formada por DAOs, objetos que ofrecen una vista orientada a objetos de una entidad, e internamente se encargan de gestionar la persistencia usando la tecnología que sea conveniente de forma transparente (sea JDBC, JPA, Hibernate, etc.).

La vista incluye los objetos encargados de presentar la interfaz al usuario. Estos componentes mostrarán al usuario el resultado de las transacciones realizadas con el modelo.

Como ya hemos dicho, si el diseño se ha realizado bien, el cambio del modelo no debe afectar a los componentes de la vista. Del mismo modo si se decide cambiar la presentación o permitir varios tipos de presentación es responsabilidad de la vista permitirlo sin necesidad de modificar el modelo.

En el desarrollo de los componentes que implementan la vista se utilizan normalmente tecnologías como *XML/XSL*, páginas *JSP*, páginas *ASP*; es decir, tecnologías distintas a las utilizadas para desarrollar el modelo y más enfocadas a permitir el diseño de interfaces gráficas de usuario.

El controlador o controladores decidirán qué vistas se muestran dependiendo de los clientes o los resultados proporcionados por el modelo. Todo esto es transparente para los componentes del modelo que únicamente responden a peticiones del controlador.

Como hemos comentado, el patrón MVC fue originalmente pensado para las interfaces de usuario de aplicaciones de escritorio, por lo que en el contexto de su aplicación al entorno web existe alguna pequeña diferencia a considerar, que hará que el mismo patrón tenga una diferente aplicación en cada caso.

De esta forma, en una aplicación Web cuando cambia el modelo no se actualiza automáticamente la vista, lo que sí ocurría en la concepción original del MVC en la que el modelo emite eventos de cambio que captura la vista y ésta se actualiza (comportamiento según patrón *Observer*).

En las aplicaciones web, el controlador se utiliza como punto central en el que se recibe las peticiones del usuario que se traducen en llamadas al modelo y a las vistas que generan la interfaz para el cliente (no por tanto como eventos, sino como invocaciones directas). Como veremos más adelante, una tecnología muy adecuada para implementar estos controladores son los *servlets*.

15. Java Platform Enterprise Edition

El lenguaje Java, que empezó siendo un lenguaje para sistemas empotrados y que saltó a la fama como lenguaje para el desarrollo de *applets*, ha pasado a ser en estos momentos un lenguaje maduro, moderno, para el que se han construido diferentes entornos de desarrollo dependiendo de los objetivos perseguidos.

Hasta 2006, Java disponía de las siguientes 3 plataformas/ediciones de desarrollo:

1. **J2ME** (*Java 2 Micro Edition*). Es la plataforma dirigida al desarrollo de aplicaciones para dispositivos como teléfonos móviles o PDA's. Como el resto de las plataformas está formada por un conjunto de APIs definidas por grupos de trabajo en los que han colaborado empresas del sector de los dispositivos y empresas de software.
2. **J2SE** (*Java 2 Standard Edition*). Consiste en el entorno de ejecución estándar de Java. Como el resto, está formado por un conjunto de APIs para crear distintos tipos de aplicaciones como *applets* que se ejecutan en el entorno del navegador y aplicaciones cliente que se ejecutan sobre una máquina virtual que se distribuye tanto con el entorno de desarrollo como con el entorno de ejecución creado para la plataforma en la que la aplicación debe funcionar.
3. **J2EE** (*Java 2 Enterprise Edition*). Consiste en un conjunto de especificaciones dirigidas a facilitar el desarrollo de sistemas empresariales en arquitecturas multicapa.

A partir de 2006, como resultado de una decisión de marketing, Sun adoptó un cambio en la nomenclatura y en el esquema de versiones de sus productos Java, de forma que se pasó del etiquetado "*Java 2*" al actual "*Java Platform*", y de las versiones numeradas como v1.X (que alcanzó hasta la v1.4) a numerar ahora como v5.0 y sucesivas.

Por lo tanto, actualmente las plataformas de desarrollo son:

1. **Java SE** o Java Platform Standard Edition
2. **Java EE** o Java Platform Enterprise Edition
3. **Java ME** o Java Platform Micro Edition

A lo largo del presente documento se utilizará el término Java EE ya que es un término ya consolidado en publicaciones, libros y documentación.

Java EE aparece con el objetivo de minimizar los problemas asociados al desarrollo de aplicaciones multicapa en Internet (entornos heterogéneos, integración de tecnologías, complejidad, servidores complejos y propietarios).

Sun Microsystems, con el apoyo de otras grandes empresas del software, impulsó una iniciativa de recopilación y elaboración de una serie de especificaciones y recomendaciones para crear una arquitectura estándar que permitiera reducir el coste y la complejidad de estos desarrollos.

Las especificaciones *Java Platform Enterprise Edition* recogen:

- 3 **Especificaciones básicas para los servidores de aplicaciones Java** que permiten la movilidad entre distintos proveedores, y por lo tanto, creación de un mercado competitivo entre las empresas de software que provoca un abaratamiento de sus productos.
- 4 Un **conjunto extenso de librerías** sobre las que poder desarrollar sistemas complejos.
- 5 Un **conjunto de manuales de buenas prácticas** que permitan una formación sólida entre los profesionales dedicados al desarrollo de este tipo de sistemas.

Las especificaciones básicas forman lo que se conoce como la plataforma Java EE, que se ha convertido en la referencia sobre la que la práctica totalidad de fabricantes de servidores de aplicaciones desarrollan sus sistemas, y con la que el primer objetivo de crear un gran mercado competitivo se ha conseguido.

Java EE 7⁶ está compuesto por los siguientes elementos:

3. SDK (Software Development Kit) de Java EE:
 4. Servidor de aplicaciones de código abierto Glassfish. Compatible con Java EE 7, es decir ha superado los test de compatibilidad. Incluye las librerías Java EE.
 5. Documentación de las APIs: Conjunto de tecnologías Java EE basadas todas ellas en JSR (Java Specification Requests).
 6. Ejemplos de código
 7. Tutorial
 8. Una implementación de referencia (Your first cup) Java EE sobre la que se pueden desarrollar prototipos y ver un ejemplo de una plataforma Java EE completamente operativa.
 9. Un conjunto de tests de compatibilidad (Java EE Compatibility Test Suite (CTS)) que permiten verificar si una plataforma (servidor) es completamente compatible con las especificaciones de la plataforma.
 10. Los *Java EE BluePrints*, es decir, un conjunto de buenas prácticas para el desarrollo de aplicaciones multicapa y con clientes ligeros.

En los siguientes apartados analizaremos los diferentes aspectos que forman la plataforma Java EE repasando

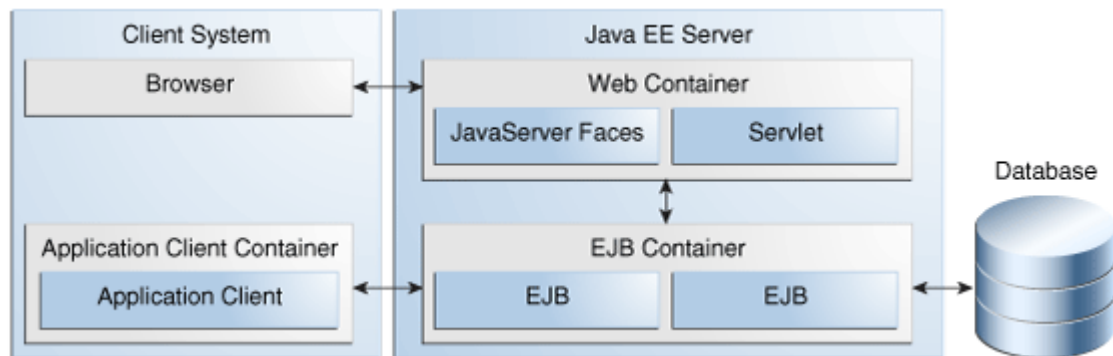
1. Arquitectura Java EE
2. Componentes Web
3. Contenedores
4. Servicios
5. Empaquetamiento y distribución de aplicaciones Java EE

Se recomienda el tutorial de Java EE 7 de la página de Oracle para ampliar cualquier información.

⁶En este tema se hará referencia a la versión más reciente existente en este momento de esta especificación, que es Java EE 7 (recordar que Oracle compra Sun en 2010). Si bien todavía no hay muchos servidores compatibles con esta versión. Actualmente lo normal es trabajar con servidores compatibles con Java EE 6 o Java EE 5.

15.1. Arquitectura Java EE

La arquitectura de Java EE se representa en el siguiente diagrama conceptual:



Como se puede observar se dice que una aplicación Java EE está formada por tres niveles o capas (three-tiered) debido a que está distribuida en tres localizaciones diferentes: la máquina cliente, el servidor Java EE y el servidor de Base de Datos, a diferencia de las arquitecturas de dos niveles o capas cliente-servidor.

La especificación Java EE distingue 3 tipos de componentes.

1. **Clientes:** se ejecutan en el cliente. Se distinguen los siguientes tipos:
 - Cliente web (web client): se llama thin client o cliente ligero, está formado por páginas web con contenido HTML, XML... que se visualizan en un navegador.
 - Cliente pesado/aplicación (application client): cliente con una interfaz gráfica más elaborada (con Swing, AWT, JavaFx) y con cierta lógica a ejecutar en la aplicación cliente. Se instala en la máquina a diferencia del cliente web que no necesita instalación (solo un navegador). Necesita tener instalado una JVM en cliente. A diferencia del cliente web, el cliente pesado interactúa directamente con componentes del contenedor EJB (Enterprise Java Beans), si bien pueden abrir conexiones HTTP e interactuar con componentes del contenedor web (aunque no es lo habitual).
 - Applets: los applets pueden ser también un cliente de una aplicación Java EE. Necesitan instalación de una JVM en la máquina

Sin lugar a duda los clientes web, son los preferidos ya que no necesitan la instalación en cliente de ninguna JVM y permiten un diseño más limpio y modular ya que separan la programación de la aplicación en Java con el diseño web, permitiendo a los programadores de la parte web la no necesidad de conocer código Java para programar el cliente web (basado principalmente en estándares de lenguajes de marcado HTML, XML, XSL etc..).

2. **Componentes web:** los componentes web son servlets o páginas web creadas mediante JavaServer Faces o JSP (páginas JSP) y se ejecutan en el servidor Java EE dentro del contenedor web. (Notar que la especificación Java EE no contempla a los applets y a los clientes web como componentes web).
3. **Componentes de negocio:** código Java dedicado a resolver el problema particular del negocio de la aplicación. Estos componentes son los EJB (Enterprise Java Beans) y se ejecutan al igual que los componentes web en el servidor Java EE pero dentro de otro contenedor, el contenedor EJB. Son los encargados de interactuar con el sistema de almacenamiento.

Si recordamos el patrón MVC podríamos decir que los contenedores de cliente y parte de los componentes del contenedor web son la vista, en el contenedor web está el controlador y en el contenedor EJB está el modelo. Evidentemente esta es una aproximación ya que puede existir parte del modelo en el contenedor web o incluso en el contenedor de aplicaciones cliente.

Mencionar que, aunque en el diagrama el último nivel se representa mediante una base de datos, Java EE se refiere a este nivel como EIS tier (Enterprise Information System), es decir sistema de información de

negocio, el cual es un concepto más amplio que el de servidor de base de datos, ya que además de una base de datos, puede ser un sistema ERP (Enterprise Resource Planning), un sistema de transacciones contra un mainframe o cualquier otro sistema existente.

A partir de este punto vamos a repasar los elementos que aparecen en el diagrama, es decir: componentes, y contenedores, para acabar viendo los servicios o tecnologías Java EE.

15.2. Componentes Java para el desarrollo web

15.2.1. Servlets

¿Qué es un servlet?

De forma resumida podemos decir que los servlets son componentes web desarrollados en Java y utilizados para generar contenido dinámico: clases Java que extienden la interfaz *HttpServlet* y que son invocadas a través de una URL pudiendo generar dinámicamente diversos tipos de contenido como texto, imágenes y sonido, aunque normalmente es texto con formato HTML o XML. Este contenido es devuelto al cliente mediante el protocolo HTTP.

La clase *HttpServlet*

La clase *HttpServlet* es una clase abstracta de la que heredarán todos los servlets destinados a dar servicios desde un servidor web. Cada servlet debe sobrecargar al menos un método que normalmente es uno de los siguientes:

- *doGet()*. Se ejecuta cuando se recibe una petición HTTP GET.
- *doPost()*. Se ejecuta cuando se recibe una petición HTTP POST.
- *doPut()*. Se ejecuta cuando se recibe una petición HTTP PUT.
- *doDelete()*. Se ejecuta cuando se recibe una petición HTTP DELETE.
- *init()* y *destroy()*. Gestionan el ciclo de vida del *servlet* ejecutándose en la creación y destrucción del *servlet*.
- *getServletInfo()*. Lo utiliza el *servlet* para proporcionar información acerca de si mismo.
- *service()*. Que maneja la petición HTTP para redirigir al método doXXX que corresponda.

Los métodos doXXX reciben como parámetros objetos de la clase *HttpServletRequest* con la información de la solicitud HTTP y *HttpServletResponse* donde se incluye la información de respuesta a la petición HTTP. En ellos se implementa normalmente la funcionalidad del *servlet* ante los distintos tipos de solicitudes HTTP.

Como ya se ha comentado en este punto, los servlets normalmente se ejecutan en servidores multihilo por lo que un *servlet* debe gestionar la concurrencia y tener en cuenta los recursos compartidos tanto en memoria (variables de clase) como en ficheros, bases de datos, etc.

¿Cómo funciona un servlet?

Un servlet se instala, gestiona y ejecuta en un componente del sistema denominado contenedor web dentro del servidor Java EE.

Más adelante en este tema veremos qué es un contenedor dentro de la plataforma Java EE, pero de momento lo que nos interesa es que este contenedor va a proporcionar al servlet independencia de la plataforma, ya que el desarrollador se podrá encargar sólo de cumplir con la especificación mientras que el contenedor se ocupará de su ejecución independientemente del entorno.

Para entender cómo funciona un servlet vamos a analizar el ciclo de vida del servlet desde su instalación hasta su desinstalación o parada.

1 El servlet es instalado en el contenedor. Básicamente esta instalación consiste en indicar en el fichero de configuración del contenedor la URL a la que responderá y la clase que implementará, y por supuesto copiar la clase en el directorio del servidor.

2 Cuando el contenedor recibe por primera vez una petición de un servlet, el contenedor crea un objeto de la clase Java que lo implementa. Al ser Java un entorno multihilo, los servlets están diseñados para permitir dar respuesta a un alto número de solicitudes concurrentes.

3 Para cada petición:

4. El cliente (normalmente un navegador web) envía la petición HTTP al servidor web.

5. El servidor web identifica la petición como una URL que está asociada con algo que se debe ejecutar en el contenedor web y reenvía la petición al contenedor.

6. El contenedor (o motor) de *servlets* tiene una configuración en la que se indica para cada URL solicitada qué objeto debe crear. Por lo tanto, construye o, si ya estuviese creado por una petición anterior utiliza, el objeto de la clase correspondiente a la petición de la URL.

7. Una vez creado el objeto, ejecuta el método *doGet()* o *doPost()* (según se indique en la petición HTTP) del objeto. Todo *servlet* tendrá estos métodos puesto que lo obliga la interfaz *HttpServlet*.

8. El *servlet* analiza la información que viene en el *request* de la petición HTTP y realiza las llamadas pertinentes a otros componentes Java, como son los objetos de negocio, de cara a obtener la información necesaria.

9. Finalmente, el *servlet* genera la respuesta en el *response* construyendo la página HTML, XML o lo que sea que el servidor web vaya a enviar al cliente.

Los *servlets* se han mostrado adecuados para determinadas funcionalidades dentro de las aplicaciones corporativas ya que tienen toda la versatilidad de un lenguaje de propósito general como Java que permite estructurar secuencias de control complejas sin obtener un código excesivamente difícil de mantener, funcionalidades como la implementación de controladores dentro de arquitecturas del tipo MVC, así como para la generación de salidas binarias (imágenes, sonido, video, etc.)

Sin embargo, desde los inicios del desarrollo con *servlets* se comprobó que, aunque los servlets eran un excelente sistema para gestionar las peticiones web de contenido dinámico, no eran un sistema cómodo para generar la interfaz HTML de la respuesta ya que había que codificar mucho para conseguir un nivel aceptable de presentación. Por ello veremos que en se han ideado otras tecnologías Java como JSP que aligeran esta fase.

15.2.2. Java Server Pages (JSP)

¿Qué es un JSP?

Como hemos explicado en el apartado anterior los servlets demostraron ser un excelente sistema de gestionar las peticiones de contenido dinámico, pero no era un sistema cómodo para generar la respuesta ya que había que codificar mucho para generar un buen nivel de presentación en HTML.

Sun Microsystems intentó resolver el problema formalizando un nuevo estándar llamado JavaServer Pages (JSP) en 1999. Siguiendo la misma filosofía de otras tecnologías de generación dinámica de páginas como Microsoft's Active Server Pages, una página JSP es un documento de texto que contiene HTML y bloques de código embebido que se ejecuta en el servidor para generar el contenido dinámico de esa página HTML. Así, se puede decir que la tecnología JSP facilita el desarrollo de páginas web con contenido estático y dinámico.

Como veremos un JSP es en realidad un *servlet* orientado a la presentación pues, como veremos, antes de ser ejecutado el motor que maneja las JSP preprocesa su contenido y las convierte en auténticos *servlets*.

Esta tecnología no es más que una aproximación a la tecnología *servlet* de una forma más natural para la creación de la parte estática de una página web.

Una página JSP puede contener código en diferentes lenguajes:

- Código para interpretar en la máquina cliente:
 1. HTML
 2. Un lenguaje interpretado por navegadores como JavaScript o VBScript.
- Código para ejecutar en el servidor:
 1. Directivas JSP
 2. Código Java insertado entre las etiquetas “<%” y “%>”.
 3. Expresiones de Java entre las etiquetas “<%= ... %>” que se ejecutan y devuelven un valor que se escribe directamente en la salida.
 4. Etiquetas JSP estándar. Son etiquetas que realizan funciones básicas del manejo de las páginas JSP como, por ejemplo, la carga de librerías, la iteración de colecciones, acceso a beans, etc. Ejemplo: `<jsp:useBean id="Bean" scope="request" class="Clase"/>`
 5. Etiquetas personalizadas (*custom tags*). Estas etiquetas son en realidad funcionalidades complejas encapsuladas en clases y publicadas en unos ficheros “.tld” para que se puedan invocar desde las páginas JSP con una sintaxis muy similar a la de las etiquetas HTML. Ejemplo: `<milibreria:mitag nombre="ejemplo" />`

La diversidad de lenguajes que se pueden agrupar en un JSP ha sido el origen de la principal crítica a esta tecnología.

Siempre se argumenta que, aunque evidentemente hacen que el contenido estático se pueda desarrollar más fácilmente, cuando existe mucho contenido dinámico y las aplicaciones tienen funciones avanzadas de configuración, personalización, etc. las páginas JSP acaban siendo una mezcla de muchos lenguajes muy difíciles de seguir por los propios desarrolladores y, por lo tanto, tienen un alto coste, no sólo de desarrollo sino sobre todo de mantenimiento.

¿Cómo funciona un JSP?

Una página JSP se puede ver como una página HTML con trozos de código Java que se ejecutan cuando se solicita la página. Pero lo que es importante entender es lo que sucede cuando se solicita una página JSP ya que, como se ha dicho más arriba, un JSP acaba siendo un *servlet*.

Existen dos fases claramente diferenciadas durante la ejecución de un JSP: la fase de generación de código y compilación del *servlet* generado y la fase de carga de la clase obtenida por la compilación del *servlet*.

- El cliente (normalmente un navegador web) hace una petición HTTP al servidor web de una página JSP.
- El servidor web identifica la petición como una URL que está asociada con algo que se debe ejecutar en el contenedor web y reenvía la petición al contenedor.
- El contenedor “sabe” que la petición de una página JSP, en realidad, es la petición del *servlet* generado a partir de la página, por lo que busca la clase compilada y generada a partir del JSP.
- Si no existe esa clase, el contenedor interpreta la JSP para generar un *servlet* que es, en ese momento, compilado.
- Una vez que encontrado ese *servlet*, el contenedor crea un objeto de la clase correspondiente a dicho *servlet* generado y compilado y, a partir de ese instante, continúa la ejecución como si fuera un *servlet* normal.

Inicialmente podríamos pensar que la ejecución de un JSP puede llegar a resultar muy lenta, ya que tenemos que realizar una generación de código y una compilación durante el tiempo que transcurre desde que se hace la petición hasta que se inicia la respuesta del sistema.

En realidad, el contenedor de JSP se encarga de optimizar este proceso realizando esas tareas solo cuando es necesario; algunos, incluso, permiten realizar esta generación y compilación en el mismo momento de la instalación con lo que incluso el rendimiento de la primera petición del usuario es exactamente igual al que tendría un *servlet*.

Directivas JSP

Las directivas JSP proporcionan instrucciones al contenedor de JSP acerca de la página JSP. Esta información la puede utilizar el contenedor para la generación del *servlet* o para la ejecución. La sintaxis básica es:

```
<%@ directiva atributo1="valor1" atributo2="valor2" ... %>
```

Algunas de las directivas existentes son:

- **page:** Permite especificar atributos asociados a la página.
- **include:** Permite incluir otro fichero en el JSP antes de la generación del *servlet*.
- **taglib:** Permite especificar la librería de etiquetas personalizadas que se utiliza en esa página.

Elementos de script

Un JSP permite incluir las siguientes categorías de etiquetas con código Java.

1. **Declaraciones:** Para incluir sentencias que declaran métodos o variables que se usarán en la página JSP. La sintaxis es `<%! declaración %>`
2. **Expresiones:** Son expresiones que se evalúan y el resultado se convierte a una cadena de caracteres que se inserta en la página JSP. La sintaxis es `<%= expresión %>`
3. **Scriptlets:** Son trozos de código Java incluidos entre las etiquetas del JSP. El código Java va entre las etiquetas `<%` y `%>`

Etiquetas JSP

Son etiquetas propias de la especificación JSP y tienen efecto en el momento de la ejecución del JSP. Algunas de estas etiquetas son:

- `jsp:useBean`
- `jsp:setProperty`
- `jsp:getProperty`
- `jsp:forward`
- `jsp:include`

Etiquetas personalizadas

Además de las etiquetas estándar de JSP, la especificación JSP permite al desarrollador generar sus propias librerías con etiquetas. Estas librerías no son más que clases Java que siguen unos determinados estándares.

Para poder utilizar etiquetas personalizadas en una página JSP se debe importar la librería mediante la directiva *taglib*:

```
<%@ taglib uri="/NombreFichero.tld" prefix="milibreria" %>
```

Donde "milibreria" es el prefijo que se utilizará para referenciar a una etiqueta personalizada. Por ejemplo:

```
<milibreria:mitag ..... />
```

15.2.3. JavaServer Faces (JSF)

JavaServer Faces (JSF) es una especificación Java EE (JSF 2.0 - JSR-314) para el desarrollo de interfaces de usuario basadas en componentes en aplicaciones web Java. JSF por sí solo no es un framework, sino una especificación Java EE. La implementación de referencia de la especificación JSF es el proyecto Oracle Mojarra, el cual es una implementación de código abierto de JSF.

JSF 2 utiliza Facelets como su sistema de plantillas por defecto. En contraste, JSF 1.x utiliza JavaServer Pages (JSP) como su sistema de plantillas por defecto.

Se amplía esta especificación en el apartado de frameworks.

15.2.4. Enterprise JavaBeans

Los Enterprise JavaBeans son componentes de servidor que encapsulan la lógica de negocio de una aplicación y se pueden ejecutar de forma distribuida. Debido a que los Enterprise beans se ejecutan en contenedores EJB, aprovechan los servicios de bajo nivel que éstos proporcionan permitiendo al desarrollador centrarse en resolver los problemas del negocio.

Los componentes EJB pueden encontrarse físicamente ubicados en máquinas distintas de las que albergan los componentes que les invocan. Para gestionar esta comunicación los componentes hacen uso del protocolo de llamada remota RMI/IIOP.

Estas operaciones de invocación remota son bastante pesadas por lo que habrá que evaluar si realmente nos interesa pagar este precio para obtener la mejora de rendimiento en otros aspectos.

Existen dos tipos de Enterprise Beans en Java EE 7: *Session* y *message-driven*. Hay que mencionar que anteriormente (antes de la especificación EJB 3.0) se contemplaba un tercero, los entity beans si bien estos han sido reemplazados por la tecnología JPA (Java Persistence API).

Session beans

Representa la transacción de un cliente en el servidor de aplicaciones Java EE. El bean desarrolla una tarea específica del negocio en el servidor. Dentro de este tipo de EJB podemos encontrar tres subtipos *Stateless*, *State Full* y *Singleton*.

- Los *stateless session beans* son componentes sin estado, es decir, no guardan información entre distintas llamadas de un cliente. Esto permite que el número de objetos dando servicio sea menor que el número de clientes y que, por lo tanto, se puedan utilizar lo que se llaman *pool* de objetos (se define un conjunto de objetos en memoria y van siendo compartidos por todas las peticiones de los usuarios).
- Los *state full session beans* son componentes que sí que mantienen el estado del cliente. Por lo tanto, este tipo de EJB se consideran muy pesados ya que tienen que mantenerse en memoria durante todo el tiempo que dura la conversación entre el cliente y el EJB.

Además, el rendimiento puede verse alterado si durante la conversación el número máximo de EJB se alcanza o se acaba la memoria; en ese caso el contenedor EJB se encarga de escribirlo en disco para poder recuperarlo cuando sea necesario lo que permitirá al sistema seguir funcionando, aunque con un rendimiento muy inferior.

Un ejemplo en su utilización puede ser para mantener durante la sesión del usuario los artículos que va seleccionando en su carro de la compra.

- Singleton sesión beans: se instancia una sola vez por aplicación y existe durante todo el ciclo de vida de la aplicación. Es compartida por todos los clientes de la aplicación. Por ejemplo: la carga de una lista de productos que es la misma para todos los clientes. En vez de que cada cliente ejecute una vez la lista de productos, esta se crea solo la primera vez y es compartida por todos los clientes de la aplicación.

Message driven beans

Los *message driven beans* actúan como *sesion beans* pero de forma asíncrona. En realidad, son una especie de componentes de tipo *listener* que se emplean para el procesamiento asíncrono de mensajes.

Este tipo de EJB se utiliza cuando las operaciones que va a realizar el EJB son muy costosas en tiempo o cuando el tiempo de respuesta es indeterminado y es mejor dar una respuesta asíncrona que tener esperando al cliente durante mucho tiempo.

15.3. Contenedores

Las especificaciones Java EE definen un contenedor como el software responsable de ofrecer una serie de servicios de ejecución a los componentes Java EE de aplicación. Estos servicios los prestan a través del conjunto de APIs que define la especificación Java EE.

Sin embargo estas llamadas a las APIs nunca se realizan directamente: se hacen siempre a través del contenedor; de este modo, el contenedor puede incluir la funcionalidad declarada en los ficheros de configuración proporcionados en la instalación de los componentes tales como gestión de transacciones, seguridad, gestión de estados y conexiones.

La especificación Java EE define 4 tipos de contenedores:

1. Contenedores de *applets*: gestiona la ejecución de los applets. Está formado por el navegador y el Java Plug-in instalado en la máquina cliente ejecutándose ambos en la máquina cliente.
2. Contenedores de aplicaciones Java en cliente: gestionan la ejecución de los componentes de las aplicaciones Java cliente, es decir las instaladas en la máquina cliente. Necesitan la instalación de un JRE en la máquina cliente. Tanto el contenedor como la aplicación se ejecutan en la máquina cliente.
3. Contenedores de componentes web: gestiona la ejecución de páginas web, servlets y algunos EJB. Los componentes web y el contenedor se ejecutan en el servidor Java EE.
4. Contenedores de EJBs: gestiona la ejecución de componentes EJB. Los componentes EJB y el contenedor se ejecutan en el servidor Java EE.

Los contenedores de applets y de aplicaciones Java no los vamos a ver en detalle ya en este tema.

15.3.1. Contenedor Web

Un contenedor web es el encargado de ejecutar los componentes web (servlets y páginas web creadas con JSF o JSPs) y de proporcionarles los servicios necesarios de seguridad, concurrencia, transacciones, implantación, etc.

Es importante no confundir un contenedor web con un servidor web: un **servidor web** está optimizado fundamentalmente⁷ para **servir páginas HTML estáticas**, mientras que un **contenedor web** actúa como una **capa intermedia de software entre el servidor web y el servlet**.

Para evitar errores, en muchos casos se prefiere hablar de **contenedores de servlets** en referencia a los contenedores web, ya que su principal función es la ejecución e implementación de Servlets, JSF y JSPs, sin implementar nada del resto de especificaciones de Java EE.

Su funcionamiento es sencillo:

1. El servidor web transmite la petición HTTP al contenedor, éste crea un objeto que implementa la interfaz *HttpServletRequest* y pasa la petición al servlet.
2. El servlet ejecuta su lógica, haciendo las llamadas oportunas a la lógica de negocio y genera una respuesta que el contenedor transmite al servidor web.
3. Por cada petición HTTP el contenedor es responsable de crear y gestionar un nuevo hilo de ejecución que sirva la petición.

Han tenido mucho éxito para el desarrollo de aplicaciones puesto que muchas de las soluciones que han surgido en la comunidad se edifican sobre Servlets y JSP (Struts o Axis, por ejemplo).

⁷ La mayoría de los servidores tienen módulos para permitir la generación de páginas dinámicas basadas en lenguajes interpretados como: ASP, PHP, Perl, etc.

La implementación de referencia de un contenedor de Servlets es Tomcat (de hecho, muchos servidores de aplicaciones cuentan con un contenedor web de servlets Tomcat en su interior).

Cualquier servidor de aplicaciones Java EE tiene un contenedor de Servlets pero además implementa el resto de especificaciones de Java EE a través del contenedor de EJBs, que explicaremos a continuación.

15.3.2. Contenedor EJB

El contenedor de EJB es el lugar donde residen y se ejecutan los EJBs instalados en un servidor de aplicaciones Java EE. Proporciona a los EJB un conjunto de servicios como caché, concurrencia, persistencia de los objetos, seguridad, gestión de transacciones, bloqueos, etc.

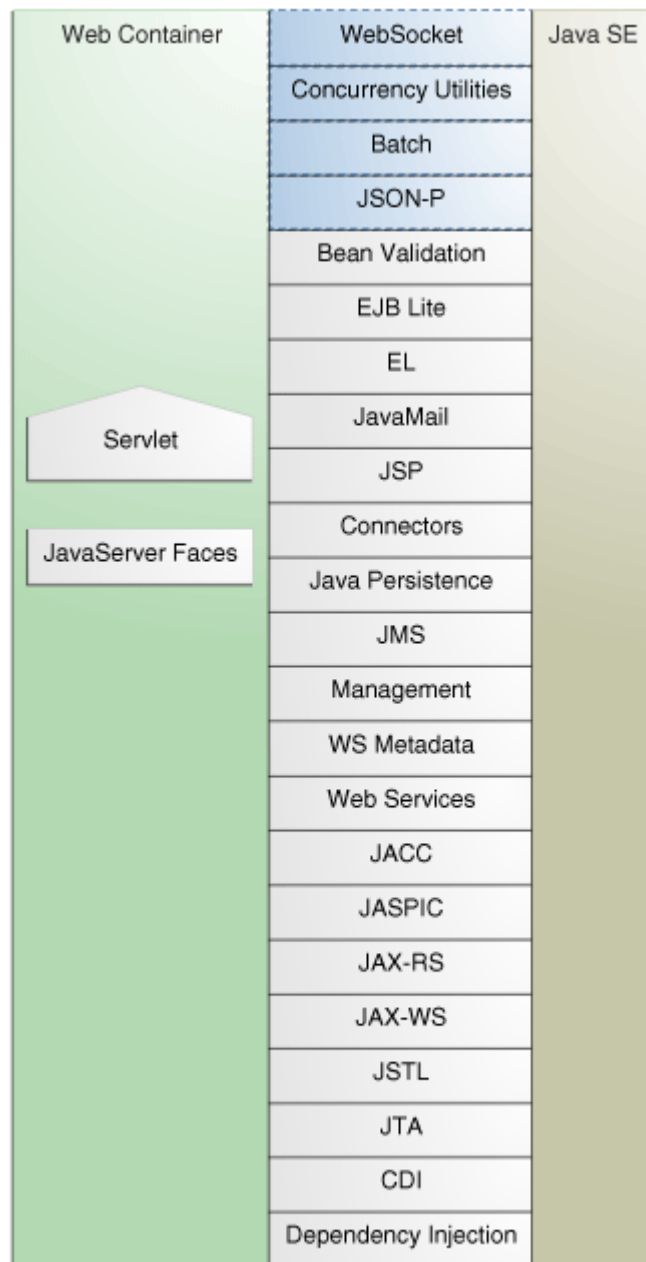
Un contenedor de EJBs se arranca automáticamente cuando se arranca el servidor de aplicaciones. En un mismo contenedor pueden residir varios EJBs.

15.4. Servicios Java EE

La plataforma Java EE define una serie de servicios o APIs que debe proporcionar un servidor de aplicaciones para facilitar y estandarizar el desarrollo de aplicaciones Java basadas en esta arquitectura y para ser certificado como servidor de aplicaciones Java EE.

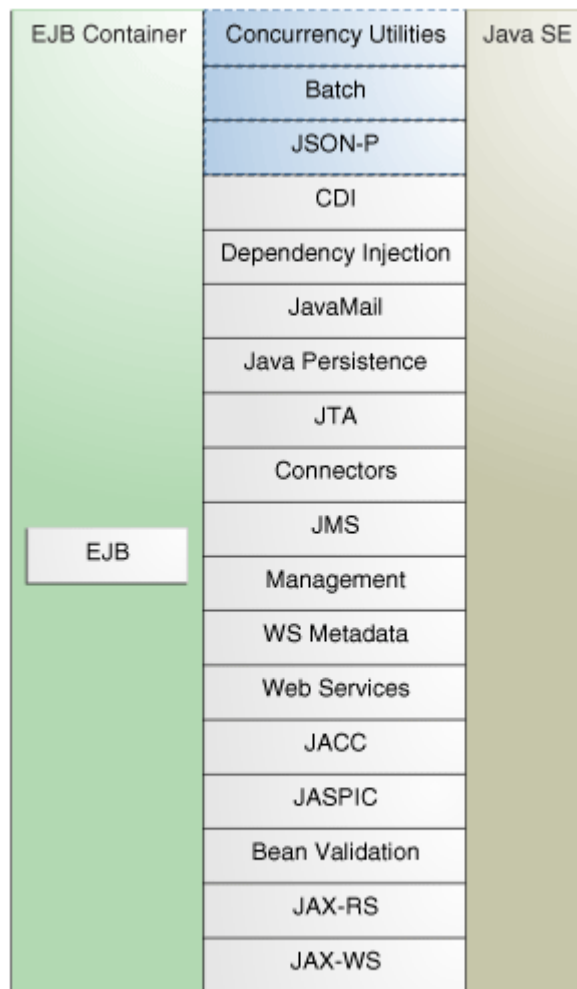
Esta serie incluye servicios como los de acceso a base de datos, gestión de mensajes asíncronos, correo electrónico, etc.

En las siguientes imágenes se pueden ver los servicios (APIs) que deben de implementar los contenedores que se ejecutan en los servidores Java EE, es decir, el contenedor web y el contenedor de EJBs.



New in Java EE 7

Java EE APIs in the web container



New in Java EE 7

Java EE APIs in the EJB container

A continuación vamos a destacar y profundizar en algunos de los más importantes.

15.4.1. Java Persistence

Java Persistence es la API estándar para la gestión de la persistencia y el mapeo objetos/modelo relacional.

Esta especificación, desarrollada por el grupo de expertos encargado de la especificación EJB 3.0, proporciona facilidades para el mapeo relacional de objetos Java para los desarrolladores de aplicaciones Java que gestionen un modelo de objetos Java para acceso y gestión de bases de datos relacionales.

Está totalmente inspirada en Hibernate, tecnología que comentaremos en el apartado 0, ya que la concepción original de persistencia de los Entity Beans fue un fracaso por su alta complejidad mientras que tecnologías como Hibernate triunfaron en paralelo y como alternativa por su sencillez y comodidad.

Java EE 7 exige compatibilidad Java Persistence API 2.1.

15.4.2. Servlet

La API dedicada a Servlet no es más que el conjunto de funciones explicado de forma genérica en el apartado de componentes Java para el desarrollo web.

Java EE 7 exige compatibilidad con Servlet 3.1.

15.4.3. JSP

Al igual que con los servlets la API de JSP es la explicada de forma genérica en el apartado de componentes Java para el desarrollo web.

Java EE 7 compatibilidad con JSP 2.3.

15.4.4. Java Message Service

La API JMS es un estándar que pretende facilitar a los componentes Java EE la creación, envío, recepción y lectura de mensajes que se intercambian entre componentes software o aplicaciones.

La utilización de mensajes permite una comunicación débilmente acoplada y asíncrona, en la que un componente emisor puede enviar un mensaje a un destino y continuar con la ejecución de la tarea, mientras que el componente receptor recogerá ese mensaje del destino. De este modo el emisor no tiene que saber nada del receptor ni viceversa. Lo único que ambos deben conocer es el formato del mensaje.

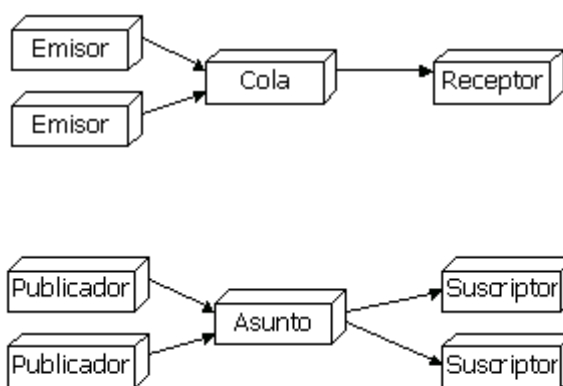
Es importante diferenciar este tipo de intercambio de información de las comunicaciones entre componentes fuertemente acoplados como RMI, que requiere que un componente conozca la interfaz que define los servicios que el componente remoto ofrece, así como de otros modelos como la comunicación a través de correo electrónico, que está más preparada para la comunicación entre personas o entre aplicaciones y personas, que entre componentes.

La API JMS minimiza los conceptos que un programador debe aprender antes de utilizar un producto de mensajería que proporcione todas las características necesarias para construir una aplicación compleja.

Además, maximiza la portabilidad de estas aplicaciones entre diferentes proveedores de productos de mensajería.

La API JMS permite comunicación asíncrona y fiable, ya que asegura que cada mensaje llega una y sólo una vez al receptor.

Existen dos modos de trabajo con mensajes: basado en colas de mensajes (*modelo point-to-point*) y basado en suscripciones (*modelo publish-subscribe*). Ambos modos están soportados por la API JMS.



Java EE 7 exige compatibilidad con JMS 2.0.

15.4.5. Java Transaction API

La API JTA está diseñado para permitir a los componentes Java EE gestionar sus propias transacciones, permitiendo que varios componentes participen en una transacción común.

Esta API la podemos dividir en dos niveles:

- Nivel de aplicación: Es utilizado por los componentes de la aplicación para indicar el inicio y fin de las transacciones.
- Nivel de gestor de transacción: Es utilizado como la interfaz entre el gestor de transacciones y el gestor de recursos utilizados en cada transacción. Es decir, entre el servidor de aplicaciones que ejecuta los componentes y los sistemas de gestión de bases de datos implicados en las transacciones.

Java EE 7 exige compatibilidad JTA 1.2.

15.4.6. Java IDL (Interface Definition Language)

La API Java IDL permite a las aplicaciones Java EE invocar a objetos CORBA externos usando el protocolo IIOP.

Estos objetos CORBA pueden estar implementados en cualquier lenguaje y típicamente están fuera del servidor de aplicaciones Java EE.

15.4.7. Java Naming and Directory Interface (JNDI)

JNDI es la API estándar para el acceso a servicios de directorio y servicios de nombres como LDAP y DNS.

Esta API consta de dos niveles:

- Un nivel de aplicación utilizado por los componentes de la aplicación para acceder a los servicios de directorio y de nombres
- Un nivel de proveedor de servicio utilizado como la interfaz para vincular a un proveedor de servicios de directorio y nombres con el servicio que ofrece.

Esta API Java EE también se encuentra incluida en la edición Java SE.

15.4.8. Java Mail Technology

La API Java Mail permite a las aplicaciones incorporar la funcionalidad de envío y recepción de mensajes de correo electrónico de una forma sencilla.

Esta API tiene dos partes una interfaz de nivel de aplicación que es la que utilizarán nuestros componentes y una interfaz de proveedor de servicio.

JavaBeans Activation Framework (JAF), está incluida dentro de la API Java Mail y se utiliza para determinar el tipo de cualquier dato encapsulando el acceso al mismo.

Se utiliza habitualmente para construir componentes JavaBean que contengan tipos de datos complejos facilitando el manejo de los mismos a los componentes o aplicaciones que los utilicen.

Java EE 7 exige compatibilidad con JavaMail 1.5.

15.4.9. JavaServer Faces

La tecnología JavaServer Faces ha sido introducida anteriormente y se explica más en detalle en el apartado de frameworks.

A continuación vamos a ver APIs Java EE pero que se incluyen también en la edición Java SE.

15.4.10.JDBC

JDBC es la API estándar para conectar programas desarrollados en Java con sistemas de gestión de bases de datos (SGBD).

Esta API permite realizar peticiones a los SGBD mediante instrucciones SQL. JDBC es muy similar en su concepción al otro estándar de acceso a bases de datos utilizado ampliamente en la industria denominado ODBC. De hecho, el primer nivel de compatibilidad de JDBC es un *bridge* de acceso JDBC-ODBC para acceder a bases de datos desde programas Java a través de drivers ODBC.

Java EE 7 exige compatibilidad JDBC 4.1.

15.4.11.Java API for XML (JAXP) y Streaming API for XML (StAX)

La API JAXP nos facilita el acceso, la generación, validación, transformación y en general el desarrollo de nuestros componentes o programas para acceder a ficheros XML, para lo que configura una API independiente de una implementación de procesador XML concreta.

Así, configura un nivel de conexión de implementaciones de proveedores ya sean de API SAX o API DOM, así como soporte de motores de transformación XSLT.

Por su parte, la API StAX expone métodos para el procesamiento de documentos XML de forma iterativa y basada en eventos.

Descrita en la especificación JSR-173, los documentos XML se tratan como una serie filtrada de eventos, y los diferentes estados se pueden almacenar en una forma procedimental, de forma que en realidad la API StAX está compuesta de dos APIs, una para el cursor y otra para el iterador.

A diferencia de SAX, con la que guarda el parecido del procesamiento por eventos, la API StAX es bidireccional, permitiendo tanto la lectura como la escritura de documentos XML en lugar de sólo la lectura como en SAX.

Otra diferencia entre ambas es la forma de *parsing* realizada, de tipo *pull* en el caso de StAX que permite que el control resida en el cliente, frente al tipo *push* de SAX, en el que es el *parser* XML el que mantiene parte del control y envía información al cliente.

JAXP es una de las APIs Java EE que también se incluyen dentro de la edición Java SE.

15.4.12.Java Architecture for XML Binding (JAXB)

JAXB proporciona una manera de asociar clases Java a representaciones XML. JAXB se puede utilizar de forma independiente o en combinación con JAX – WS.

Todos los contenedores Java EE cliente de aplicación, contenedores web y contenedores EJB soportan el API JAXB.

La plataforma Java EE 7 requiere JAXB 2.2.

15.4.13.Java API for XML Web Services (JAX-WS)

JAX-WS proporciona soporte para servicios web que utilizan la API JAXB para asociar clases Java a XML. Define un estándar para asociar métodos Java a operaciones WSDL.

La plataforma Java EE 7 requiere JAX - WS 2.2.

15.4.14.SOAP with Attachments API for Java (SAAJ)

SAAJ permite a los desarrolladores Java generar y recoger mensajes según la especificación SOAP y SOAP con ficheros adjuntos que pueden ser documentos XML o ficheros en formato MIME. Es una API de bajo nivel ya que otras APIs como JAX-WS la utilizan.

15.4.15.JavaBeans Activation Framework (JAF)

La API JAF proporciona un marco de trabajo para manejar y gestionar información con diferentes tipos MIME, originada en diferentes formatos y localizaciones.

Una de las APIs que utiliza intensivamente la API JAF es API JavaMail, motivo por el que se incluye dentro de la plataforma Java EE.

15.4.16.Java Authentication and Authorization Service (JAAS) y Java Authorization Service Provider Contract for Containers (JACC)

El API JAAS proporciona un sistema de autenticación y control de acceso de usuarios y grupos para una aplicación.

Su función principal es proporcionar independencia entre los sistemas básicos de autenticación y control de acceso utilizados y los mecanismos que las aplicaciones basadas en Java EE utilizan para dar acceso a las diferentes funcionalidades.

Así, por ejemplo, permite que una aplicación gestione las funciones reservadas a los administradores de la misma, independientemente de si la autenticación estaba basada en Kerberos, usuario y password o una tarjeta criptográfica.

Por su parte, el API JACC define un contrato entre un servidor de aplicaciones Java EE y un proveedor de servicios de autorización, proporcionando independencia del sistema y mecanismos de autorización así como del proveedor del servicio de autorización y las aplicaciones Java EE.

15.4.17.Arquitectura de Conectores Java EE

La Arquitectura de Conectores Java EE (*Java EE Connector Architecture*) es utilizada, fundamentalmente por integradores de sistemas, para la creación de enlaces entre diferentes sistemas de información con sistemas en arquitectura Java EE.

Esta arquitectura define una Interfaz de Proveedor de Servicios (*Service Provider Interface* o SPI) que habilita a los componentes Java EE para interactuar con otros sistemas de información de la organización como los sistemas integrados de gestión empresarial (ERP), procesadores de transacciones de *mainframes* (p.e CISC), etc.

Cada sistema de información que quiera conectarse requiere de un componente Java EE que implementa la tecnología de conexión para ese sistema.

Estas implementaciones se denominan *resource adapters* y serían equivalentes a un driver JDBC, ya que ambos proporcionan un API para el acceso a una aplicación externa al servidor Java EE.

Entre el servidor Java EE y el *resource adapter* se establecen una serie de acuerdo sobre el servicio ofrecido, que incluyen:

- Un contrato sobre la gestión de la conexión que permite que el servidor Java EE disponga de un pool de conexiones al sistema de información correspondiente, permitiendo un entorno de

aplicación escalable capaz de soportar un gran número de clientes accediendo al sistema de información de la organización.

- Un contrato de gestión de transacciones entre el gestor de transacciones del servidor J2EE/Java EE y el sistema de información, que soporte el control de la transaccionalidad en el acceso a los recursos del sistema de información.
- Un contrato de seguridad que permite un acceso seguro al sistema de información, reduciendo las amenazas de seguridad y protegiendo la información y recursos de valor gestionados por el sistema de información.
- Un contrato de gestión de hilos de ejecución que permite al *resource manager* delegar tareas en otros hilos de ejecución y que el servidor de aplicaciones maneje un pool o conjunto de hilos de ejecución con contextos de seguridad y transaccionalidad independientes.
- Un contrato que permite a los *resource adapters* la entrega de mensajes a los *message driven beans* independientemente del estilo de mensaje específico, la semántica del mensaje o la infraestructura de mensajería utilizada para la entrega.
- Un contrato que permite a un *resource adapter* propagar un contexto de transacción al servidor de aplicaciones Java EE de forma que sus interacciones con el servidor y componentes de aplicaciones son parte de la transacción de importación del contexto.
- Un contrato opcional que proporciona una interfaz genérica de comandos entre las aplicaciones y los *resource adapters*.

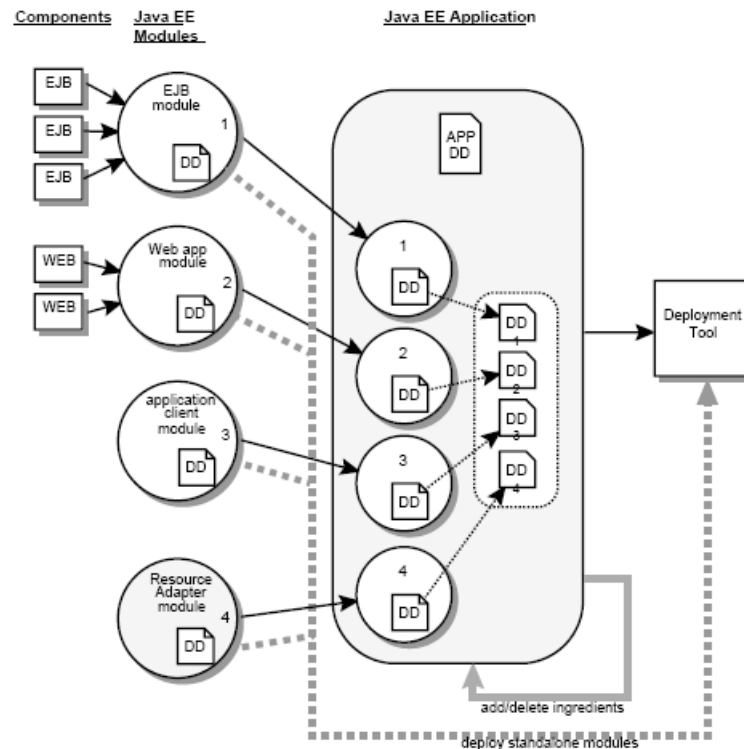
15.5. Empaquetamiento e instalación de aplicaciones Java EE

Los componentes Java EE deben ser empaquetados para facilitar su posterior instalación en el servidor Java EE.

La especificación Java EE define cómo se debe realizar este empaquetado de modo que cualquier servidor compatible con las especificaciones pueda instalar el paquete en su seno.

Una aplicación Java EE está compuesta, en principio, por componentes EJB, componentes web (servlets, JSPs, JSFs), clases con utilidades de servidor, componentes de cliente y ficheros estáticos como HTML y GIF.

Todos estos componentes se deben empaquetar junto con unos ficheros de configuración conocidos como descriptores de despliegue o *deployment descriptors*.



15.5.1. Java Archive (.jar)

Los ficheros en formato JAR están basados en el formato ZIP y se utilizan para empaquetar un conjunto de ficheros.

Estos ficheros contienen clases Java, EJB, imágenes, etc. y ficheros de metainformación que se pueden utilizar para la configuración de las clases incluidas en el paquete.

15.5.2. Web Archive (.war)

Los ficheros WAR y EAR se utilizan para instalar una aplicación web en un servidor de aplicaciones compatible con las especificaciones Java EE.

El objetivo de estos dos tipos de ficheros es empaquetar en un solo fichero todos los componentes necesarios para ejecutar una aplicación, consiguiendo que la instalación de una aplicación empaquetada de este modo sea muy fácil.

El empaquetado de ficheros se hace, al igual que para los ficheros JAR, mediante el formato ZIP.

Un fichero WAR representa una aplicación web e incluye todos los componentes que la aplicación necesita para su ejecución:

- Librerías Java. Clases Java de utilidades.
- Servlets.
- JSPs
- Ficheros HTML
- Otros recursos que pueda necesitar la aplicación (ficheros de imagen, sonido, plantillas, etc.)

- Un *deployment descriptor*. Fichero de configuración con el nombre `web.xml` que debe seguir el formato definido por el dtd proporcionado por J2EE. Normalmente en este fichero se define la correspondencia que permite la ejecución de los servlets de la aplicación.

15.5.3. Enterprise Archive (.ear)

El concepto de fichero EAR está básicamente asociado a los EJB, aunque también se utiliza como mecanismo para aplicaciones web.

Un fichero EAR puede contener:

- Uno o más ficheros WAR con el contenido definido más arriba.
- Uno o más `ejb-jars` que contendrían los EJB de la aplicación.
- Uno o más ficheros JAR con las clases que requiera la aplicación.
- Un *deployment descriptor*: un fichero en el que se define cómo se quiere instalar la aplicación con los parámetros adecuados para la utilización de los servicios de los contenedores. A partir de la especificación 1.3 de J2EE el estándar proporciona un formato de fichero a través de un dtd.

15.6. Servidores de aplicaciones Java EE

Podemos definir a un servidor de aplicaciones Java EE como un conjunto de elementos de software que implementan los contenedores de servidor definidos en las especificaciones Java EE, una implementación de las APIs de la misma y un conjunto de herramientas que permiten configurar y gestionar el conjunto de elementos.

15.6.1. Tests de compatibilidad

Oracle proporciona, junto con cada versión de la especificación, un conjunto de pruebas de compatibilidad (CTS. Compatibility test suite).

Este conjunto de pruebas permite asegurar la compatibilidad entre los diferentes servidores de aplicaciones de distintos vendedores, ayudando a garantizar que las aplicaciones y componentes desarrollados para la plataforma Java EE podrán ser instalados en cualquier servidor de aplicaciones que haya superado las pruebas.

El TCK para la plataforma Java EE es conocido como la suite de pruebas de compatibilidad (CTS) y está formado por un paquete que proporciona un conjunto de pruebas, herramientas y documentación para obtener una garantía de compatibilidad.

15.6.2. Servidores de aplicaciones del mercado

Existen un buen número de servidores de aplicaciones compatibles con Java EE. Estos servidores compiten en un mercado en el que la funcionalidad básica está definida por las especificaciones Java EE y que, por lo tanto, enfocan sus elementos diferenciadores hacia factores como:

1. Rendimiento.
2. Precio.
3. Fiabilidad.
4. Escalabilidad.
5. Herramientas de ayuda a la gestión y la configuración de los servidores tanto en tiempo de desarrollo como en tiempo de explotación de los sistemas desarrollados bajo estas plataformas.

Existe un gran número de servidores que han pasado los tests de compatibilidad. Centrándonos en este caso en las versiones Java EE 7 y Java EE 6, que Oracle publica en su web la siguiente lista de servidores compatibles con la especificación.

Servidores compatibles con Java EE 7 a fecha de la redacción del tema (septiembre de 2014):

1. Glassfish Server 4.0
2. TMAX JEUS 8
3. WildFly 8.x

Servidores compatibles con Java EE 6 a fecha de la redacción del tema (septiembre de 2014):

1. Glassfish Server 3.x
2. TMAX JEUS 7
3. Apache Geronimo 3.0
4. JBOSS Application Server 7.x
5. IBM WAS (Webshpere Application Server) 8.x
6. Oracle Weblogic Server
7. Fujitsu Interstage Application Server 10.1
8. ...

Como puede observarse en las dos listas, aunque en la actualidad (septiembre de 2014) la última versión publicada de Java EE es la 7, todavía hay muchos servidores que no han sido certificados como compatibles, por lo que en la actualidad se trabaja con las ediciones Java EE 5 y 6 principalmente.

16. Frameworks de desarrollo web en Java

Una vez hemos visto y analizado la especificación Java EE como uno de los máximos exponentes de las aplicaciones web basadas en Java es interesante pasar a analizar los *frameworks* de desarrollo Java más populares de los últimos años.

Estos *frameworks* han potenciado enormemente el desarrollo de aplicaciones web basadas en Java frente a otras tecnologías como ASP, PHP o Python, además de constituir, en muchos casos, una alternativa a la propia arquitectura Java EE. A nivel profesional se puede decir que casi siempre se utiliza algún framework de desarrollo. Desarrollar una aplicación Java EE sin hacer uso de ningún framework de desarrollo está orientado más a nivel educativo o aplicaciones profesionales no muy complicadas o que parten del requisito de no utilizar ninguno y desarrollar solo con las APIs Java EE.

La ubicua existencia de las arquitecturas de desarrollo multicapa en el entorno web ha supuesto que nazcan *frameworks* orientados a las diferentes capas, apareciendo un mayor número de *frameworks* para dar solución a la capa de presentación y el modelo MVC, y en menor medida, *frameworks* con vocación de solución de la capa de persistencia de datos.

En los próximos apartados, junto a una definición de carácter general sobre qué se entiende bajo el concepto de *framework* de desarrollo, se analizarán sólo algunos de los *frameworks* más conocidos y destacados.

16.1. ¿Qué es un framework de desarrollo?

Un *framework* es un **conjunto de bloques de construcción de software** que los programadores pueden usar, extender o personalizar **para construir sus soluciones informáticas específicas**, de forma que no tienen que empezar desde cero cada vez que quieran escribir una aplicación.

Es, por tanto, una infraestructura de soporte que generalmente incluye programas, librerías y un lenguaje de *scripting*, para ayudar al desarrollo de un proyecto software.

Un *framework* representa una **completa arquitectura de software** que proporciona una estructura y una metodología de trabajo, diseñados ambos con el intento de facilitar el desarrollo de software y permitir a los diseñadores y programadores pasar más tiempo identificando requisitos software que tratando con los tediosos detalles de bajo nivel para alcanzar un sistema funcional.

De esta forma, por ejemplo, un equipo que usa Apache Struts para desarrollar un sitio web de un banco puede enfocarse en cómo los retiros de ahorros van a tener que funcionar, en lugar de preocuparse de cómo se controla la navegación entre las páginas en una forma libre de errores.

No obstante, aunque su propósito sea el de facilitar el desarrollo, no han faltado siempre quejas acerca de que el uso de *frameworks* añade código innecesario (por hacer un símil, sería como usar un programa de construcción HTML en vez de construir el HTML en texto plano), en muchas ocasiones no orientado a alcanzar las mejores prestaciones, y que la preponderancia de *frameworks* competitivos y complementarios ha supuesto que el tiempo que se pasaba programando y diseñando, es ahora “gastado” en aprender a usar dichos *frameworks*.

En los próximos apartados, veremos un breve esbozo de los *frameworks* de desarrollo de aplicaciones web en Java más conocidos y utilizados, en concreto:

- Apache Struts
- Spring Framework
- Hibernate ORM
- JSF
- Tapestry

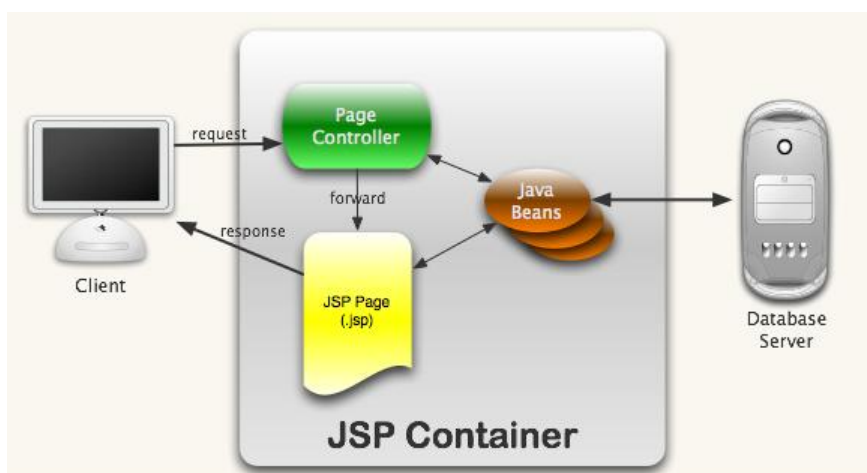
Aunque se muestran las principales características de los frameworks a día de hoy, no pretende ser una descripción detallada y precisa de ellos, sino una visión general.

Si un equipo de desarrollo de una unidad TIC está pensando implementar una aplicación web en Java, se encontrará ante la previsible elección de uno o varios *frameworks* de desarrollo para realizarlo. Antes de empezar cualquier proyecto de desarrollo se debe evaluar todas las tecnologías posibles y su adecuación o no a los requisitos de la aplicación o sistema a desarrollar.

16.2. Apache Struts

Apache Struts apareció en escena en 2000 como un *framework* de código abierto para el desarrollo de aplicaciones web Java EE, año desde el que ha disfrutado una exitosa existencia colaborando en la construcción de algunos, y siendo uno de los principales *framework* de las aplicaciones web desarrolladas en Java a día de hoy.

Desde su nacimiento, Apache Struts proporcionó un sólido marco de trabajo que **organizaba adecuadamente la combinación de JSPs y Servlets** en el desarrollo de aplicaciones web, según lo que por aquel entonces se llamaba arquitectura Modelo 2, y que simplemente consistía en delegar el papel del Controlador a los Servlets y la Vista a las páginas JSP, como se muestra en la siguiente figura.



El objetivo de Struts era, por tanto, bastante sencillo: **separar limpiamente el modelo** (lógica de la aplicación que interactúa con la base de datos) **de la vista** (páginas HTML presentadas al cliente) y **del controlador** (instancia que pasa información entre vista y modelo y controla el flujo de ejecución).

Para Struts, el controlador simplemente es un servlet llamado *ActionServlet* (extiende de *HttpServlet*) y el entorno facilita la escritura de plantillas para la vista o la capa de presentación, generalmente escrita en forma de JSPs o bien mediante XML/XSLT (con posible integración de otros *frameworks* como Velocity).

El programador web en Struts es responsable de escribir el código del modelo, y crear la configuración central de la aplicación en un fichero llamado *strutsconfig.xml* que vincula al modelo, la vista y el controlador.

Las **peticiones del cliente se envían al controlador en forma de “Acciones”** (clases "*Actions*") definidas en el fichero de configuración, de forma que el servlet controlador las recibe, las redirige hacia la correspondiente clase *Action*, que es la que finalmente interactúa con el código específico del modelo.

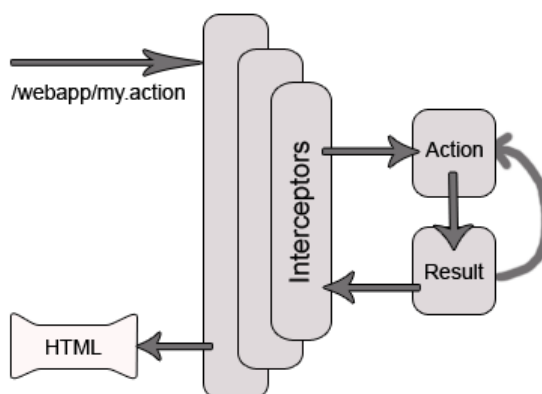
El código que implementa el modelo devuelve una cadena de texto que le indica al controlador qué página de salida debe enviar al cliente, pasando la información entre el modelo y la vista a través de *JavaBeans*.

Para evitar embeber código Java en las páginas JSP, una librería de etiquetas específica (*custom tag library*) permite leer y escribir el contenido de estos *beans* en la capa de presentación, quedando de esta forma un código limpio y mantenible.

Tras una rápida expansión, la relativa complejidad que alcanzaban las clases “*Action*” junto a ciertos problemas de extensibilidad y de inclusión de novedades como soporte para AJAX, provocó que surgieran varias líneas de trabajo para mejorar Struts, entre ellas Struts Tiles.

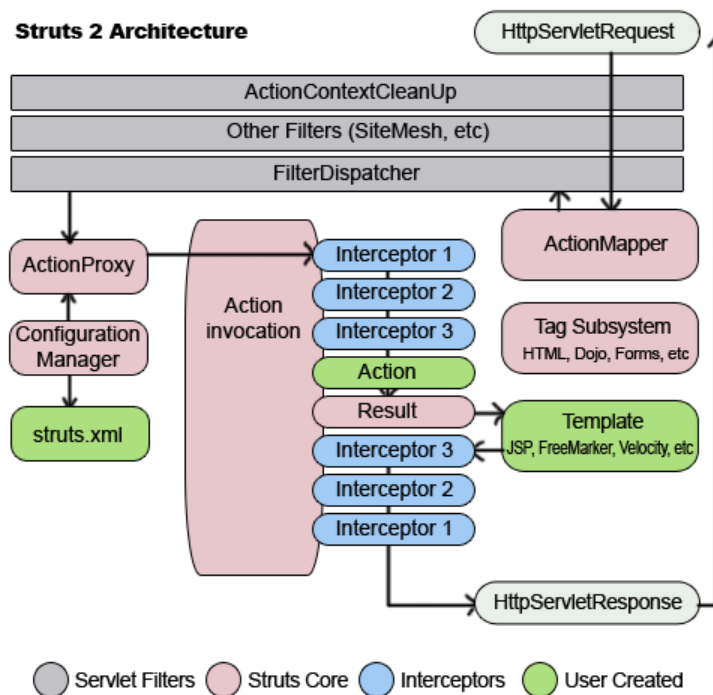
En diciembre de 2005, Struts decidió unir fuerzas con otro *framework* Java EE bastante popular, WebWork, que de hecho surgió como otra de las líneas de evolución de Struts.

La “unión” de Struts 1 y WebWork ha sido finalmente adoptada como la versión 2.0 de Apache Struts, que ha mejorado en extensibilidad, soporte de AJAX, mejoras para desarrollo ágil, etc., si bien su arquitectura no ha cambiado susceptiblemente respecto de Struts 1, y a alto nivel es la que se muestra en la siguiente figura.



A grandes rasgos, no hay grandes cambios respecto al modelo de Struts 1 exceptuando que desaparecen los *ActionForwards* y se sustituyen por *Results*.

Los mayores cambios afectan a la simplificación de las clases *Action* evitando el encorsetamiento en su desarrollo, la necesaria extensión de clases base abstractas, y a un diseño más limpio y efectivo basado en interceptores con un mejor manejo de los filtros, lo que es apreciable si aumentamos el detalle en la figura anterior obteniendo la figura siguiente donde se muestra la arquitectura Struts 2.



La versión actual (septiembre de 2014) de Struts es la 2.3.16 y mantiene la misma arquitectura mostrada en el diagrama. En el diagrama se puede ver como una solicitud (**HttpServletRequest**) pasa a través de una serie de filtros opcionales (**ActionContextCleanUp**) hasta llegar al filtro requerido **FilterDispatcher** el cual consulta el **ActionMapper** para determinar si se debe o no invocar una acción. Si el **ActionMapper** determina que sí debe de invocarse una acción, el **FilterDispatcher** delega el control al **Action Proxy**, el cual consulta al **Configuration Manager** (cargado con los datos a partir del archivo **struts.xml**). Después el **ActionProxy** crea una **ActionInvocation** la cual es responsable de invocar los interceptores necesarios antes de invocar a la acción. Una vez la acción termina es responsable de buscar el resultado apropiado asociado con el código devuelto por la acción (este mapeo se ha configurado en el fichero **struts.xml** para saber que hacer para cada código devuelto por la acción). El resultado se ejecuta entonces, haciendo uso de una plantilla JSP o FreeMarker.

Explicado todo este complejo proceso, lo importante es que el equipo del proyecto en lo que se tiene que centrar es de configurar el **struts.xml**, programar la acción de negocio correspondiente y de elaborar la plantilla de la vista correspondiente.

16.3. Spring Framework

Spring es un *framework* de código abierto para el desarrollo de aplicaciones en Java cuya primera versión fue escrita por Rod Jonhson, quien lo lanzó primero con la publicación de su libro “*Expert One-on-One Java EE Design and Development*” (Wrox Press, octubre 2002).

El *framework* fue lanzado bajo la licencia Apache 2.0 en junio de 2003 siendo la versión actual (en septiembre de 2014) la 4.1.0.

A pesar de que Spring no obliga a usar un lenguaje de programación en particular, existiendo de hecho una versión para la plataforma .NET llamada Spring.net, se ha popularizado en la comunidad de programadores Java al considerársele como una opción para sustituir o complementar el modelo de Enterprise JavaBean (EJB) de la propia especificación Java EE.

Por su diseño, el *framework* ofrece mucha flexibilidad a los desarrolladores Java así como muy bien documentación y fáciles de uso y de aprendizaje. Además, este *framework* puede emplearse en cualquier aplicación hecha en Java, es decir, no sólo es aplicable al desarrollo de aplicaciones web sino también al de aplicaciones en entorno cliente.

Sus características principales son:

- **Gestión de la Configuración basada en JavaBeans**, aplicando principios de **Inversión de Control**, especialmente la técnica de Inyección de Dependencias. Esta característica reduce las dependencias entre los componentes y las implementaciones específicas de otros componentes.
- Una **factoría de beans** en el núcleo del framework, utilizable a nivel global.
- Un **nivel de abstracción genérico para la gestión del acceso a las bases de datos**.
- Estrategias genéricas para **JTA** y una **única fuente de datos JDBC**, lo que elimina la dependencia del soporte transaccional del entorno Java EE.
- **Integración con frameworks de persistencia** como Hibernate, JDO, iBATIS o db4o.
- Un **framework propio para aplicaciones web** basadas en el **modelo MVC**, construido sobre la funcionalidad básica de Spring, y que soporta numerosas técnicas para la generación de la Vista como JSP, FreeMarker, Velocity, Tiles, iText, etc.
- Framework de **programación basada en aspectos** (*aspect-oriented programming*, AOP) para proporcionar servicios como la gestión de transacciones. Al igual que con la Inversión de Control, esta característica mejora la modularidad de los sistemas creados mediante el framework.

Para ofrecer estas características, Spring se ha conformado como pequeños *frameworks* o *módulos* diseñados en su mayor parte para trabajar de forma independiente unos de otros, proporcionando sus mejores funcionalidades cuando colaboran:

- *Framework* para el Contexto de la Aplicación (*Spring Core*): constituye el elemento principal que permite la Inversión de Control y la configuración de los componentes de la aplicación y el ciclo de vida de los objetos Java.
- *Framework* para Programación Orientada a Aspectos (*Spring AOP*): establece la infraestructura principal para este tipo de programación que establece asuntos y conceptos transversales a los diferentes módulos de una aplicación (como, por ejemplo, el *logging*).
- *Framework* para el Acceso a Datos y Gestión Transaccional (*Spring DAO & Spring ORM*): ofrece el soporte para trabajar con bases de datos relacionales utilizando JDBC, JDO, JTA así como diversos *frameworks* y herramientas de mapeo objetos-relacional (Hibernate, iBATIS)
- *Framework MVC* (*Spring Web & Spring MVC*): surgido como reacción a Struts, Spring cuenta con su propia implementación del modelo MVC, que presenta mejoras para enlace con otros frameworks, y un mejor entorno para pruebas.
- *Framework* para Acceso Remoto y Gestión Remota (*Spring Context*): abstracción para trabajar con varias tecnologías basadas en RPC sobre la plataforma Java (RMI, CORBA e incluso SOAP) tanto para conectividad cliente como exportación de objetos en servidores, y configuración remota vía JMX.
- *Framework* para Autenticación y Autorización: procesos de autenticación y autorización que dan soporte a los estándares y tecnologías más populares de la industria, así como a los diversos protocolos y frameworks de seguridad como Acegi.
- *Framework* para Mensajería: registro configurable de colas de mensajes vía JMS.
- *Framework* para Pruebas: clases de apoyo para escribir pruebas unitarias y de integración.

El éxito de Spring ha provocado que el trabajo en torno al mismo haya sido enorme, y hayan surgido progresivamente nuevas extensiones y módulos completando funcionalidades.

Como ejemplo paradigmático nos encontramos con Spring WebFlow (SWF), un *framework* situado por encima de Spring que permite la definición y representación de flujos de interfaces de usuario en las aplicaciones web de una forma clara y simple. Mediante Spring WebFlow, se definen los flujos que conforman los diálogos con el usuario, de tal forma que las respuestas a los eventos producidos por el usuario durante el diálogo van guiando la ejecución del código de la aplicación.

Spring WebFlow supone un mayor nivel de abstracción que Spring MVC, aportando así más herramientas a los programadores a la hora de implementar sus aplicaciones web.

16.4. Hibernate ORM

La mayoría de las aplicaciones necesitan trabajar con información o datos generalmente almacenados de forma persistente, pero que durante la ejecución de la aplicación se manejan y gestionan como una red de objetos en memoria.

Escribir el código necesario para realizar la tarea de obtener la información y ponerla en forma de objetos es una tarea tediosa y un foco de posibles errores, pudiendo incluso representar un esfuerzo no despreciable del desarrollo de toda la aplicación.

La tarea de realizar la persistencia de los objetos Java en una base de datos relacional se ha realizado a lo largo del tiempo mediante diferentes técnicas y herramientas (directamente con JDBC, usando *pools* de conexiones, gestores de persistencia programados según necesidad).

La funcionalidad principal de todas ellas pasa por serializar los objetos Java jerárquicamente estructurados en memoria e introducirlos en una base de datos con estructura tabular, y a la inversa desde la base de datos a su representación en memoria, siendo esencial la optimización en términos de velocidad y eficiencia de la traducción entre objetos Java y columnas y registros de la base de datos.

Este panorama ha fortalecido la tendencia de creación de herramientas que automaticen este proceso en lugar de dejarlo en manos de los programadores, llegando a la culminación del mismo cuando se trabaja con bases de datos relacionales, en cuyo caso se han creado numerosas **herramientas de mapeo relacional/objetos** (herramientas O/RM u ORM – object-relational mapping)

La variedad de herramientas de estas características es enorme, desde costosas herramientas comerciales, a la propia especificación Java EE, que puede considerarse en cierto sentido una forma de lograr la persistencia ORM. En muchos casos, sin embargo, la herramienta introduce su propia complejidad y fuerza al desarrollador a aprender reglas detalladas de cómo usarla, modificando incluso las clases de negocio de las aplicaciones para ajustarse a las necesidades del nuevo sistema.

Dentro de este contexto, **Hibernate ORM** surge como un *framework open-source* de desarrollo Java que **proporciona mecanismos de traducción o mapeo relacional-a-objetos** de forma que permite definir cómo los objetos Java se almacenan, actualizan, borran, o se extraen de una base de datos. Hibernate no sólo se encarga de realizar la **traducción entre clases Java y tablas de la base de datos** sino también de los **tipos de datos Java a tipos de datos SQL**.

Además, Hibernate ofrece facilidades de consulta y obtención de datos que pueden optimizar los esfuerzos ya realizados en entornos SQL y JDBC y reducir significativamente los tiempos de desarrollo de otra forma invertidos en la codificación manual del código SQL o JDBC necesario.

Con Hibernate, se libera al desarrollador del 95% de las tareas de programación de la persistencia de datos, ya que se le libera de crear las llamadas SQL y de la conversión manual entre el *Result Set* de datos obtenido y los objetos Java, manteniendo la aplicación portable para todas las bases de datos SQL con poca sobrecarga en términos de prestaciones.

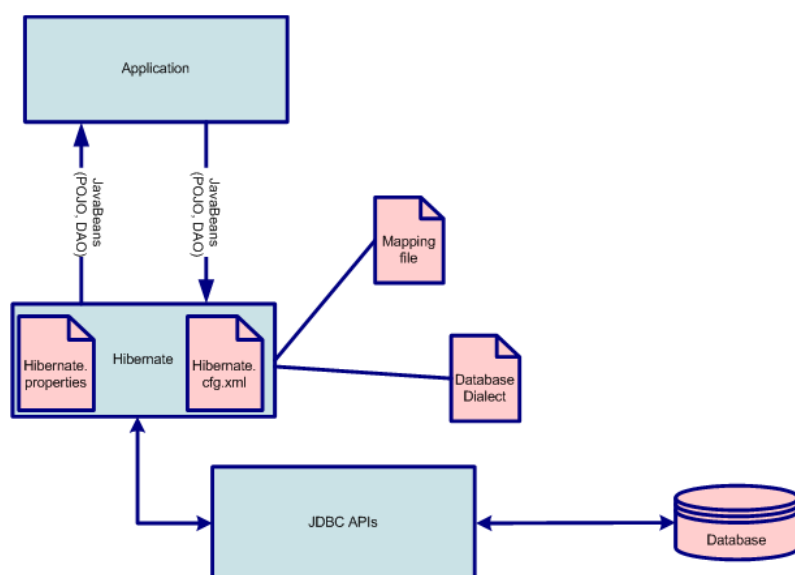
Una de las principales características de Hibernate es que no requiere que los desarrolladores implementen interfaces propietarios o extiendan clases base para conseguir persistencia de los datos. En lugar de eso, **Hibernate se apoya en la reflexión de clases Java** y en una potente librería de **generación de código Java en tiempo de ejecución** llamada Javassist (en versiones anteriores utilizaba CGLIB pero se decidió cambiar a Javaassist).

A diferencia de otras tecnologías como los EJBs-JPA, Hibernate proporciona persistencia de forma transparente para cualquier objeto Java “plano”, es decir, para cualquier POJO (*Plain Old Java Object*), lo que supone que pueda ser usado no sólo en aplicaciones web sino también en aplicaciones Java de entorno cliente.

Así, para utilizar Hibernate como mecanismo de persistir información en una base de datos, sólo es necesario escribir simples objetos POJO (*Plain Old Java Objects*), crear un fichero XML con el mapeo/transformación entre las tablas de la base de datos y las clases y sus atributos y llamar a unos pocos métodos de las APIs de Hibernate para la carga y almacenamiento de los objetos persistentes.

En tiempo de ejecución, Hibernate lee dicho documento y dinámicamente construye las clases Java con la información extraída de la base de datos.

A grosso modo, la arquitectura de Hibernate es la representada en la siguiente figura.



Para entender el escenario, consideremos una aplicación web que recoge información personal de usuario y la almacena en una base de datos, y que también posee una página que muestra la información de los usuarios almacenada.

Para implementar esta solución, podríamos almacenar los datos obteniendo los parámetros de la petición que se realiza al usuario y simplemente ponerlos en una llamada SQL de inserción. De forma similar, a la hora de mostrar información simplemente seleccionaríamos los registros deseados de la base de datos y los mostraríamos a través de una página dinámica que itera sobre el conjunto de registros.

En la mayoría de los casos, el programador desarrollará JavaBeans que representen las entidades de la base de datos, y luego desarrollará un Gestor de Persistencia que pueda operar sobre los JavaBeans para realizar determinadas operaciones de consulta, actualización, almacenamiento, búsqueda, etc., de forma que no tengamos que escribir código relativo a la persistencia directamente en nuestro código central de la aplicación.

Con Hibernate, se consigue la traducción uno-a-uno de los atributos de las clases con las columnas de una entrada de base de datos, sin necesidad de desarrollar el código SQL necesario ni el Gestor de Persistencia, y sólo teniendo que mantener los JavaBeans comentados que representan la información a almacenar.

Hibernate fue una iniciativa de un grupo de desarrolladores dispersos alrededor del mundo liderados por Gavin King. Poco tiempo después, JBoss Inc. (empresa comprada por Red Hat) contrató a los principales desarrolladores de Hibernate y trabajó con ellos brindando soporte al proyecto.

La rama de desarrollo de Hibernate es la 3.x, incorporó nuevas características, como una nueva arquitectura Interceptor/Callback, filtros definidos por el usuario, y —opcionalmente— el uso de Anotaciones para definir la correspondencia en lugar de los archivos XML o juntamente con ellos. Hibernate 3 también guarda cercanía con la especificación EJB 3.0 (aunque apareciera antes de la publicación de dicha especificación por la Java Community Process) y actúa como la espina dorsal de la implementación de EJB 3.0 en JBoss.

La versión actual (septiembre de 2014) de Hibernate es la 4.3.6, la cual soporta JPA 2.1 y sigue distribuido bajo la licencia GNU LGPL.

16.5. JSF – JavaServer Faces

JavaServer Faces (JSF) es una especificación Java EE (JSF 2.0 - JSR-314) para el desarrollo de interfaces de usuario basadas en componentes en aplicaciones web Java. JSF por sí solo no es un framework, sino una especificación Java EE. La implementación de referencia de la especificación JSF es el proyecto Oracle Mojarra, el cual es una implementación de código abierto de JSF.

JSF 2 utiliza Facelets como su sistema de plantillas por defecto. En contraste, JSF 1.x utiliza JavaServer Pages (JSP) como su sistema de plantillas por defecto.

Al tratarse de una especificación, existen distintas implementaciones de la misma. La implementación de referencia como hemos indicado es el proyecto Oracle Mojarra si bien la más utilizada, además libre, es posiblemente la implementación de la fundación Apache, MyFaces, pero podríamos optar por la solución de IBM incluida con WebSphere o cualquier otra.

A nivel práctico, se puede decir que una implementación de JSF proporciona fundamentalmente:

- Un API que define el *framework* (ciclo de vida, eventos, conversores y validadores, acceso al mundo exterior, etc.)
- Un conjunto de componentes por defecto (el valor de JSF viene con la posibilidad de usar componentes de terceros) que usaremos a través de una librería de etiquetas.
- Una librería de etiquetas que permiten interactuar con el API anterior.
- Un lenguaje de expresiones (EL) similar al existente en JSTL/JSP 2.0.

16.6. Apache Tapestry

Creado inicialmente por Howard Lewis Ship y adoptado posteriormente por la Apache Software Foundation, **Tapestry** es otro de los frameworks de desarrollo de aplicaciones web que implementa el modelo Modelo-Vista-Controlador, que usa una aproximación modular enfatizando la simplicidad.

Tapestry es un framework para el desarrollo de aplicaciones web **basado en componentes** y escrito en Java. Tapestry no usa JSPs ni ningún otro motor de plantillas, sino que **define un modelo de componentes** que pueden ser utilizados **para constituir la interfaz visual de la aplicación web**. En Tapestry todo son componentes.

Los componentes son piezas autónomas y reusables de código. Para usar uno simplemente necesitarás conocer sus parámetros todo lo demás como archivos Javascript, hojas de estilo y mensajes localizados son proporcionados por el componente. Los componentes pueden lanzar eventos y diferentes contenedores comportarse de diferente forma ante el mismo evento.

El "mantra" de Tapestry es "objetos, métodos y propiedades". Tapestry se ocupa de las tareas directamente relacionadas con la API Servlet: peticiones, respuestas, sesiones, atributos, parámetros, URL's, etc, proporcionando al programador una API de alto nivel de forma que éste puede centrarse en la lógica de negocio de la aplicación.

Esto no quiere decir que la API Servlet no está accesible al programador, sino que no es necesaria, y que el framework realmente actúa como capa intermedia entre la aplicación desarrollada y el contenedor de Servlets.

Las aplicaciones web en Tapestry se descomponen en **páginas y componentes**. Las primeras determinan la disposición de los segundos, que son los que se encargan de ofrecer la funcionalidad. De esta forma, las aplicaciones se transforman en porciones de código que se encargan respectivamente de manejar las interacciones de usuario, la lógica de negocio y la persistencia de los datos.

Los componentes, por su parte, se descomponen más aún en tipos básicos como son la validación de formularios, elementos de interfaz y objetos persistentes, y tipos suplementarios que definen como un grupo de componentes básicos interactúan con otros del grupo.

Hasta la versión 5, todos los componentes de Tapestry así como las páginas (que no dejan de ser unos componentes "especiales") están formados por:

1. Una **especificación**: la especificación es un documento XML en el que se indica el nombre del componente, qué parámetros y propiedades posee y de qué tipo son, por qué componentes está formado y cómo se conectan los parámetros de dichos componentes.
2. Una **plantilla**: la plantilla es un fichero XHTML y está formada íntegramente por etiquetas HTML. La comunicación entre el “*backend*” Java y el “*frontend*” de objetos XHTML se realiza a través de un lenguaje de programación conocido como OGNL (*Object-Graph Navigation Language*), diseñado para realizar fácilmente las operaciones *get* y *set* de las propiedades de los objetos Java. Es un lenguaje embebido en el código HTML como ocurre con PHP, ASP y JSP, pero mediante mecanismos diferentes a éstos al no utilizar etiquetas especiales para encapsular variables y código.

Tapestry, por tanto, no añade ninguna etiqueta especial, por lo que ésta puede ser editada con cualquier editor de HTML, y únicamente añade el atributo *jwcid* (Java Web Component Id) que indica que la etiqueta en la que aparece representa a un componente. Por tanto, en la plantilla se define la apariencia visual del componente, situando los componentes contenidos en él en la posición deseada.
3. Una **implementación**: cada componente tiene una clase que implementa su comportamiento, conectando propiedades a esta clase y ejecutando los métodos adecuados cuando se produzcan los eventos.

Tapestry se ha mantenido relativamente similar a lo largo de las diferentes versiones con la excepción de la versión 5 que ha sido reescrita completamente.

El desarrollo de la versión 5 comenzó al poco de ser liberada la versión 4, debido a que los desarrolladores de la última versión recién liberada se dieron cuenta de que la arquitectura original no resolvía problemas algunos problemas acuciantes.

Entre estos problemas se incluían la compleja arquitectura de componentes y la persistencia de datos, así como la necesidad de recompilar, desplegar y rearrancar sus servlets cada vez que se realizara un cambio en la aplicación. A ello, había que sumar la tendencia de las últimas versiones a no conservar la compatibilidad hacia atrás.

Frente a la versión anterior, la nueva versión 5 elimina algunas limitaciones conocidas como:

- Los componentes ya no extienden de las clases base.
- Las clases de los componentes ya no son abstractas, de forma que los componentes pasan a ser simples y puros POJOs (*Plain Old Java Objects* – Antiguos Planos Objetos Java).
- Tapestry deja de usar los archivos XML para la especificación de página y componentes. La información que solía incluirse en dichos archivos ahora se proporciona directamente en la clase Java a través de las anotaciones Java.
- Los cambios a las plantillas y clases de los componentes Tapestry son ahora recogidos inmediatamente sin necesidad alguna de reinicio. Esta característica incluso funciona en el entorno de producción, no sólo en desarrollo.
- El nuevo código base funciona considerablemente más rápido que en versiones anteriores. Los caminos críticos han sido simplificados y el uso de la reflexión ha sido virtualmente eliminado.

Para ello, Tapestry 5 ha desarrollado su propio contenedor IoC (Inversion of Control) frente a Tapestry 4 que utilizaba la tecnología HiveMind. Este contenedor propio es la pieza clave de Tapestry 5 y proporciona la Inversión de Control necesaria para evitar la necesidad de usar XML como mecanismo de comunicación, y evita tener que redespigar y reiniciar el contenedor de servlets.

La versión actual (septiembre de 2014) de Tapestry es la 5.3.7 es compatible 100% en todas las futuras versiones a través de las anotaciones Java, sin serlo con la versión 4 y anteriores y está licenciada bajo la licencia Apache License 2.0.

17. Certificaciones profesionales Java EE

Oracle tiene un amplio programa de certificaciones dirigido a desarrolladores experimentados de aplicaciones Java EE. Estas certificaciones pretenden garantizar que la persona que obtiene la certificación tiene los conocimientos y habilidades necesarios exigidos en la certificación.

Un equipo de desarrollo TIC es importante que cuente con personas con habilidades y conocimientos contrastados. Una forma de garantizarlo es exigiendo que el personal cuente con la certificación Oracle necesaria.

Dentro del gran mundo de las certificaciones Java EE destacan las siguientes:

- [Oracle Certified Expert, Java EE 6 Enterprise JavaBeans Developer](#)
- [Oracle Certified Expert, Java EE 6 Java Persistence API Developer](#)
- [Oracle Certified Expert, Java EE 6 JavaServer Faces Developer](#)

17.1. [Oracle Certified Expert, Java EE 6 Web Component Developer](#)

- [Oracle Certified Expert, Java EE 6 Web Services Developer](#)
- [Oracle Certified Master, Java EE 6 Enterprise Architect](#)

Si además se va a utilizar algún framework de desarrollo, los propios frameworks ofrecen certificaciones en sus productos. Es interesante también ver los perfiles y certificaciones que podrían ser necesarios para el desarrollo de la aplicación.

18. Casos de éxito en la Administración General del Estado

Dentro de la Administración General del Estado, como no podía ser de otra manera, se utiliza en muchos casos la tecnología Java y concretamente la edición Java EE para desarrollar aplicaciones web necesarias para responder a las necesidades de los diferentes departamentos y organismos.

Se muestran aquí una serie de proyectos relevantes desarrollados en Java EE:

1. **Pros@:** Pros@ es un marco tecnológico de la Gerencia Informática de la Seguridad Social (GISS) basado en **Java EE**, que sirve para la **construcción y ejecución de aplicaciones de gestión de la Seguridad Social (aplicaciones de la Secretaría de Estado de la Seguridad Social)**. Este marco tecnológico proporciona a todos los equipos de desarrollo una Arquitectura de Desarrollo común, una Arquitectura de Ejecución común, Aplicaciones de negocio comunes y un sistema de Promoción de las aplicaciones común.

La arquitectura de desarrollo proporciona a su vez un Framework de desarrollo propio (el framework surgió por el año 2000 debido a que no había en el mercado frameworks lo suficientemente útiles para la organización, sobre todo por no contar con la posibilidad de atacar Natural/Adabas. Este framework establece una estructura/arquitectura interna común para todas las aplicaciones, funcionalidades comunes como mail, pdf, impresión, acceso a base de datos, trabajos batch...) y Herramientas de Desarrollo (Pros@EDI – plugin para IBM Rational Software Architecture que permite crear un proyecto web Pros@).

La arquitectura de ejecución proporciona una serie de aplicaciones (Portal Pros@ - sirve para ubicar, lanzar y dotar de la misma apariencia a todas las aplicaciones) y servicios (motor de procedimientos/workflow común, seguridad, confidencialidad, auditorías, multicanalidad) que soportan la ejecución de aplicaciones Pros@.

Aplicaciones de negocio comunes a todas las aplicaciones son accesibles y compartidas: Sistema propio de permisos de seguridad (Silcon), Gestor Documental, firma electrónica, envío de SMS, impresión, log, correo electrónico...

El sistema de Promoción de aplicaciones es común para todas las aplicaciones existiendo los mismos entornos (desarrollo, integración, pre-explotación, formación y producción) y las mismas normas de calidad y de pruebas funcionales y de rendimiento.

El objetivo de este marco tecnológico establecido en la GISS es evitar:

- la heterogeneidad en los desarrollos (es decir que cada equipo o proyecto desarrolle en una tecnología diferente)
- las interfaces de usuario diferentes
- el mal aprovechamiento de los recursos: no se aprovecha lo ya inventado o desarrollado por otros equipos de desarrollo, con lo que los tiempos de desarrollo aumentaban.

En definitiva, evitar que cada proyecto “reinvente la rueda”.

Las ventajas que se persigue para los equipos de desarrollo son varias:

- misma arquitectura de las aplicaciones: se separa claramente la parte de modelo, vista y controlador y se proporcionan las clases base para cada parte del MVC.
- más sencillo de desarrollar al facilitar muchas funcionalidades comunes y ya probadas. Lógicamente existe una curva de aprendizaje del framework de desarrollo como existe en el uso de cualquier framework.
- se minimizan errores y ciclos de prueba, al reutilizar muchas funcionalidades ya probadas.
- Abstracción del tipo de BBDD al proporcionar el framework las APIs para cada tipo de BBDD (Tamino, Oracle, Adabas).

2. **Plataforma de validación de firma electrónica @firma:** @firma es una plataforma de validación y firma electrónica multi-PKI, que se pone a disposición de las Administraciones Públicas, proporcionando servicios para implementar la autenticación y firma electrónica avanzada de una forma rápida y efectiva. Por ello, el Ministerio de Hacienda y Función Pública ofrece la plataforma de servicios de validación y firma electrónica multi-PKI @firma, como un servicio de validación de certificados y firmas electrónicas, desacoplado de las aplicaciones.

El objetivo es comprobar que el certificado utilizado por el ciudadano es un certificado válido y que no ha sido revocado y que por tanto sigue teniendo plena validez para identificar a su propietario. Los servicios de la plataforma son aplicables a todos los certificados electrónicos cualificados publicados por cualquier proveedor de servicio de certificación supervisado por el Ministerio de Industria Turismo y Comercio en España, incluidos los certificados del DNle.

La plataforma de validación del Ministerio de Hacienda y Función Pública funciona como un servicio no intrusivo, que puede ser utilizado por todos los servicios telemáticos ofrecidos por las distintas Administraciones Públicas, tanto estatal, como autonómica o local. Para facilitar la integración con el servicio se proporcionan unas librerías de integración ‘Integr@’, que también permiten firma en servidor.

Además de ofrecerse como servicio, está disponible como software para instalar por las administraciones públicas (modelo federado), con múltiples utilidades de valor añadido, entre las que se encuentran la generación y validación de firmas electrónicas en múltiples formatos.

19. Referencias

Java EE Oracle	http://www.oracle.com/technetwork/java/javaee/overview/index.html
Java Blueprints Oracle	http://www.oracle.com/technetwork/java/index-jsp-136701.html
Java EE Compatibility Test Suite	http://www.oracle.com/technetwork/java/javaee/overview/compatibility-jsp-136984.html
Oracle	http://www.oracle.com
Monica Pawlan (Sun Microsystems)	Writing enterprise applications with Java2 SDK Enterprise Edition
Open Source Web Frameworks in Java	http://java-source.net/open-source/web-frameworks
Open Source Persistence Frameworks in Java	http://java-source.net/open-source/persistence
Apache Struts	http://struts.apache.org/
Spring Framework	http://projects.spring.io/spring-framework/
Hibernate ORM	http://hibernate.org/orm/
Tapestry	http://tapestry.apache.org/
JavaServer Faces Technology (JSF)	http://www.oracle.com/technetwork/java/javaee/jaserverfaces-139869.html
MyFaces	http://myfaces.apache.org/
Thinking in Java (Bruce Eckel)	http://www.bruceeckel.com
Compiladores nativos	http://gcc.gnu.org/java http://www.excelsior-usa.com/jet.html http://www.bearcave.com/software/java/comp_java.html
Programación Java en Linux (Meyers)	Editorial Prentice Hall 2002
JavaWorld	http://www.javaworld.com/
Excelsiorcompiler	http://www.excelsior-usa.com/articles/java-to-exe.html
Wikipedia	http://www.wikipedia.org/